

BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Electrical and Computer Engineering Department

COMPUTER ORGANIZATION AND MICROPROCESSOR

ENCS 2380

Project Report

SINGLE CYCLE PROCESSOR DESIGN

Prepared by:

Hiba Hmidan 1230454

Rama Anabtawi 1230338

Nancy Samhan 1230926

Instructor: Dr. Abualseoud Hanani

Section: 2

Date: 10/06/2025

Abstract

In this project, we designed and implemented a RISC 32-bit processor with 24- bit instructions. The processor is a single cycle processor, which executes one instruction in each clock cycle. The project main objective was to apply what we learned in the computer organization course, so that we build the processor from scratch using digital logic on Logisim software. The processor dealt with a total of 28 instructions divided into three types: R-type, I-type, and J-type. Each type covered different operations including arithmetic, logical, and branching operations. It also had sixteen 32-bit registers, but R0 is always hardwired to zero. The Datapath of the processor contained all the functional units which are necessary to build an instruction set architecture and ensure correct flow between different components. It consisted of RAM memory, ROM memory, ALU file, Register File, main control unit, PC controller, as well as other simple components such as adders and multiplexers. To ensure correctness of our design, we developed different test cases which covered all instructions, some while loops, if statements, and procedure calls. The project successfully achieved course objectives of having a deep understanding of processor designs, as well as allowing future processor design improvements.

Table Of Contents

1. Design and Implementation 1
   - 1.1 Register File 2
   - 1.2 ALU (Arithmetic and Logic Unit): 5
   - 1.3 Data Memory 7
   - 1.4 Instruction memory 8
   - 1.5 PC control 9
     - 1.5.1 Condition Unit 9
     - 1.5.2 PC Selector 10
   - 1.6 Controlling PC in Datapath 11
   - 1.7 The Main Control Unit 12
2. Simulation and Testing 15
   - 2.1 Instruction Set 1: (R-type instructions) 15
     - 2.1.1 Simulating the First Two instructions: 17
     - 2.1.2 Simulate the Third instruction: 18
     - 2.1.3 Simulating the Fourth instruction: 19
     - 2.1.4 Simulating the Fifth instruction: 19
     - 2.1.5 Simulating the Sixth instruction: 20
     - 2.1.6 Simulating the Seventh instruction: 21
     - 2.1.7 Simulating the Eighth instruction: 22
     - 2.1.8 Simulating the Ninth instruction: 23
     - 2.1.9 Simulate the tenth instruction: 23
   - 2.3 Instruction Set 2: (I-type instructions) 25
     - 2.2.1 Simulating the eleventh instruction: 25
     - 2.2.2 Simulating the twelfth instruction: 26
     - 2.2.3 Simulating the thirteenth instruction: 26
     - 2.2.4 Simulating the fourteenth instruction: 27
     - 2.2.5 Simulating the fifteenth instruction: 28
     - 2.2.6 Simulating the sixteenth instruction: 29

2.3 Simulating instruction set 3 30
2.3.1 Simulating the seventeenth instruction 30
2.3.2 simulating the nineteenth instruction 31
2.3.3 Simulating the 21st and the 22nd instructions 31
2.3.4 Simulating the 26th instruction 34
2.3.5 Simulating the 27th instruction 34
2.3.6 Simulating the 28th instruction 35
2.4 Testing a simple while loop program 36
2.4.1 First iteration: 39
2.4.2 Second iteration: 40
2.4.3 Third iteration: 41
2.4.4 Fourth iteration: 42
2.5 Testing a procedure and calling it one or more times 43
2.5.1 Instruction #1: 44
2.5.2 Calling Instruction #4: 45
2.5.3 Instruction #5 46
2.5.4 Calling Instruction #2 47
2.5.5 Instruction #3 48
2.6 Program Termination 48
3. Issues, limitations, and improvements 49
4. Teamwork 50
5. Conclusion 50

Table of figures

Figure 1.1:Overall Datapath implementation. 1
Figure 1.2:The Register File of the SCP. 2
Figure 1.3:Writing Operation on Register File 3
Figure 1.4:Reading Operation on Register File. 4
Figure 1.5:ALU File 5
Figure 1.6: ALU block in datapath 6
Figure 1.7:Data Memory. 7
Figure 1.8:Instruction memory. 8
Figure 1.9:PC controllers in datapath 9
Figure 1.10:PC Condition Unit 9
Figure 1.11:PC Condition block 9
Figure 1.12:PC selector 10
Figure 1.13: PC selector block 10
Figure 1.14:PC controller in Datapath 11
Figure 1.15: The Control Unit 12
Figure 1.16: Control Unit in Datapath 12
Figure 2.1:initial data in RAM 15
Figure 2.2: initial instructions in ROM 16
Figure 2.3: initial register file 16
Figure 2.4: First Instruction in Instruction Memory. 17
Figure 2.5: Second Instruction in Instruction Memory. 17
Figure 2.6: Result of Load Operation to R1 and R2 18
Figure 2.7: Third Instruction in Instruction Memory. 18
Figure 2.8: Result of R1 after AND operation 18
Figure 2.9: Fourth Instruction in Instruction Memory. 19
Figure 2.10: R4 after CAND operation 19
Figure 2.11: Fifth Instruction in Instruction Memory. 20
Figure 2.12: R4 after OR operation 20
Figure 2.13: Sixth Instruction in Instruction Memory. 20
Figure 2.14: R5 after XOR Operation 21
Figure 2.15: Seventh Instruction in Instruction Memory. 21
Figure 2.16: R5 after ADD operation 22
Figure 2.17: Eighth Instruction in Instruction Memory. 22

Figure 2.18:R4 after NADD Operation 22
Figure 2.19: Ninth Instruction in Instruction Memory. 23
Figure 2.20:R4 after SEQ Operation 23
Figure 2.21: Tenth Instruction in Instruction Memory. 24
Figure 2.22: R4 after SLT Operation 24
Figure 2.23:Instruction 11 25
Figure 2.24:R8 after ANDI 25
Figure 2.25:Instruction 12 26
Figure 2.26: R4 after CANDI 26
Figure 2.27 :Instruction 13 in Memory 27
Figure 2.28:R9 after ORI 27
Figure 2.29:Instruction 14 in Memory 27
Figure 2.30:*R9 after XORI* 28
Figure 2.31:Instruction 15 in Memory 28
Figure 2.32: R12 after ADDI 28
Figure 2.33: Instruction 16 in Memory 29
Figure 2.34:R11 after NADDI 29
Figure 2.35: Instruction SLL Simulation 30
Figure 2.36: Instruction 20 in Memory 31
Figure 2.37: R10 and R7 after SRA Operation 31
Figure 2.38: Instruction 21 in Memory 32
Figure 2.39: Values of Registers in BEQ 32
Figure 2.40: Continue Instruction Sequence 32
Figure 2.41: R1 and R2 values 33
Figure 2.42: Executing the next instruction 33
Figure 2.43: Instruction 26 34
Figure 2.44: Result of Jump Instruction 34
Figure 2.45: Instruction 27 35
Figure 2.46: R15 after JAL instruction 35
Figure 2.47: R12 value for Store instruction 35
Figure 2.48: Value 0x42000060 stored in RAM 35
Figure 2.49:instruction 1 from the while loop program 37
Figure 2.50: R1 set to 3 37
Figure 2.51: instruction 2 in simple while loop program 37
Figure 2.52: R2 set to 0 38
Figure 2.53: instruction 3 in simple while loop program 38
Figure 2.54:instruction 4 in simple while loop program 39
Figure 2.55: R2 incremented by 1 39

Figure 2.56: instruction 5 in simple while loop program 39
Figure 2.57: going back to BEQ instruction 39
Figure 2.58: going back to instruction 4 40
Figure 2.59: R2 is increased for the second time 40
Figure 2.60: reaching jump instruction 40
Figure 2.61: checking the BEQ condition again 41
Figure 2.62: executing ADD instruction again 41
Figure 2.63: R2 increased for the third time 41
Figure 2.64: reaching jump instruction again 41
Figure 2.65: checking the BEQ condition for the last time 42
Figure 2.66: exit from the while loop 42
Figure 2.67: Executing first jump instruction in the procedure call instruction set 44
Figure 2.68: Calling instruction #4 45
Figure 2.69: R2 and R4 values 45
Figure 2.70: R13 after executing instruction #4 45
Figure 2.71: Executing instruction #5 46
Figure 2.72: Calling instruction #2 47
Figure 2.73: R14 after executing instruction #2 47
Figure 2.74: Executing instruction #3 48
Figure 2.75: Termination Instruction 48
Figure 3.1: Improvement Design 49
Figure 4.1: Division of work among team 50

List of Tables

Table 1.1:ALUop description 6
Table 1.2:Control Unit Output signals 13
Table 1.3: Instructions Control Signal Encoding 14
Table 2.1: Instruction set 1 15
Table 2.2: Instruction Set 2 25
Table 2.3: Instruction Set 3 30
Table 2.4: while loop program instructions 36
Table 2.5: Instruction set for procedure calls 43

1. Design and Implementation

In this project, we used Logisim software to model and test a 32-bit processor with 24-bit instructions, which supports 16 general-purpose registers (R0 to R15) and supports three instruction formats: R-type, I-type, and J-type instructions, which involve arithmetic and logical operations, as well as branching and jumping.

Figure 1.1 shows overall Datapath implementation.

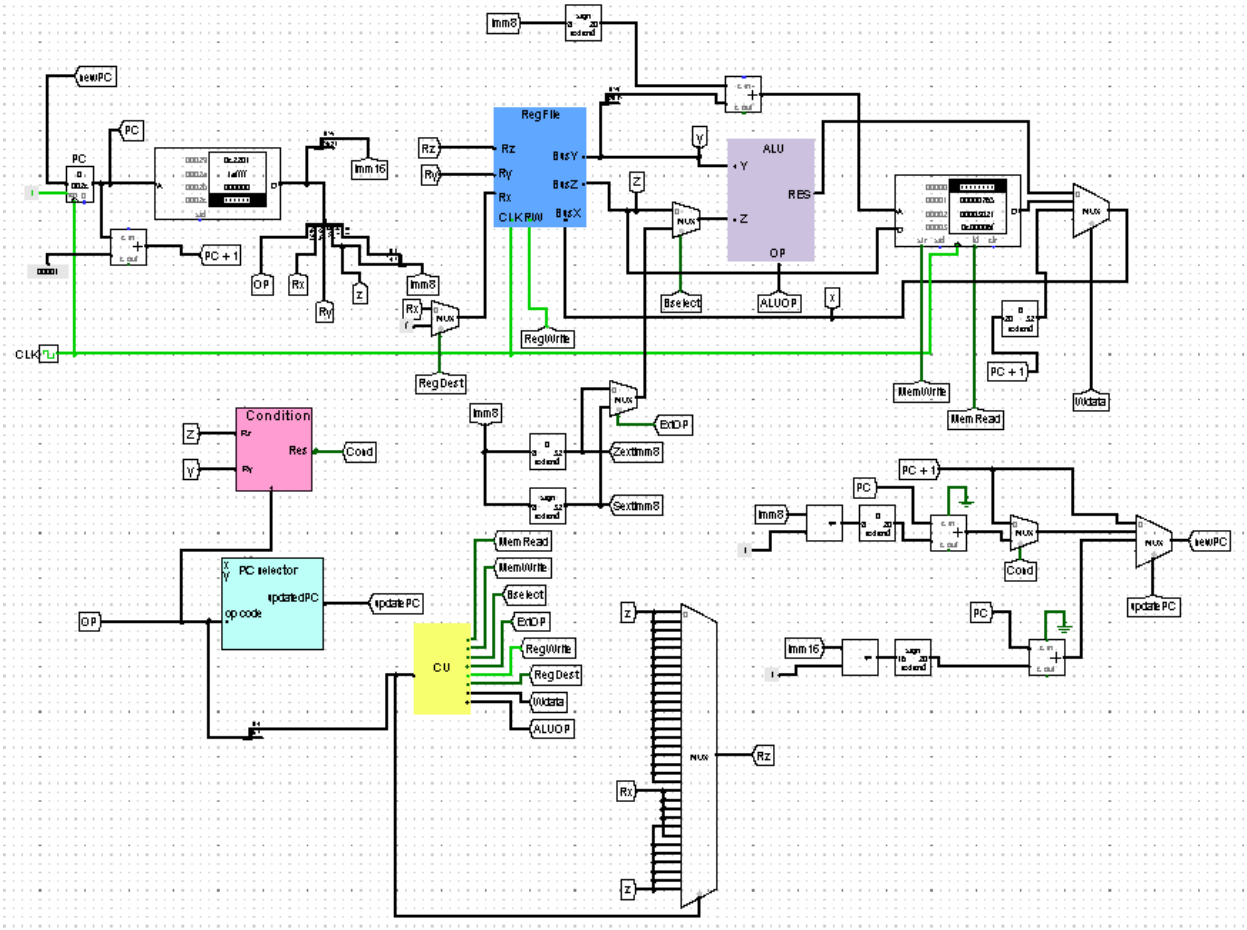


Figure 1.1:Overall Datapath implementation.

The main components of the single cycle processor design are:

1.1 Register File

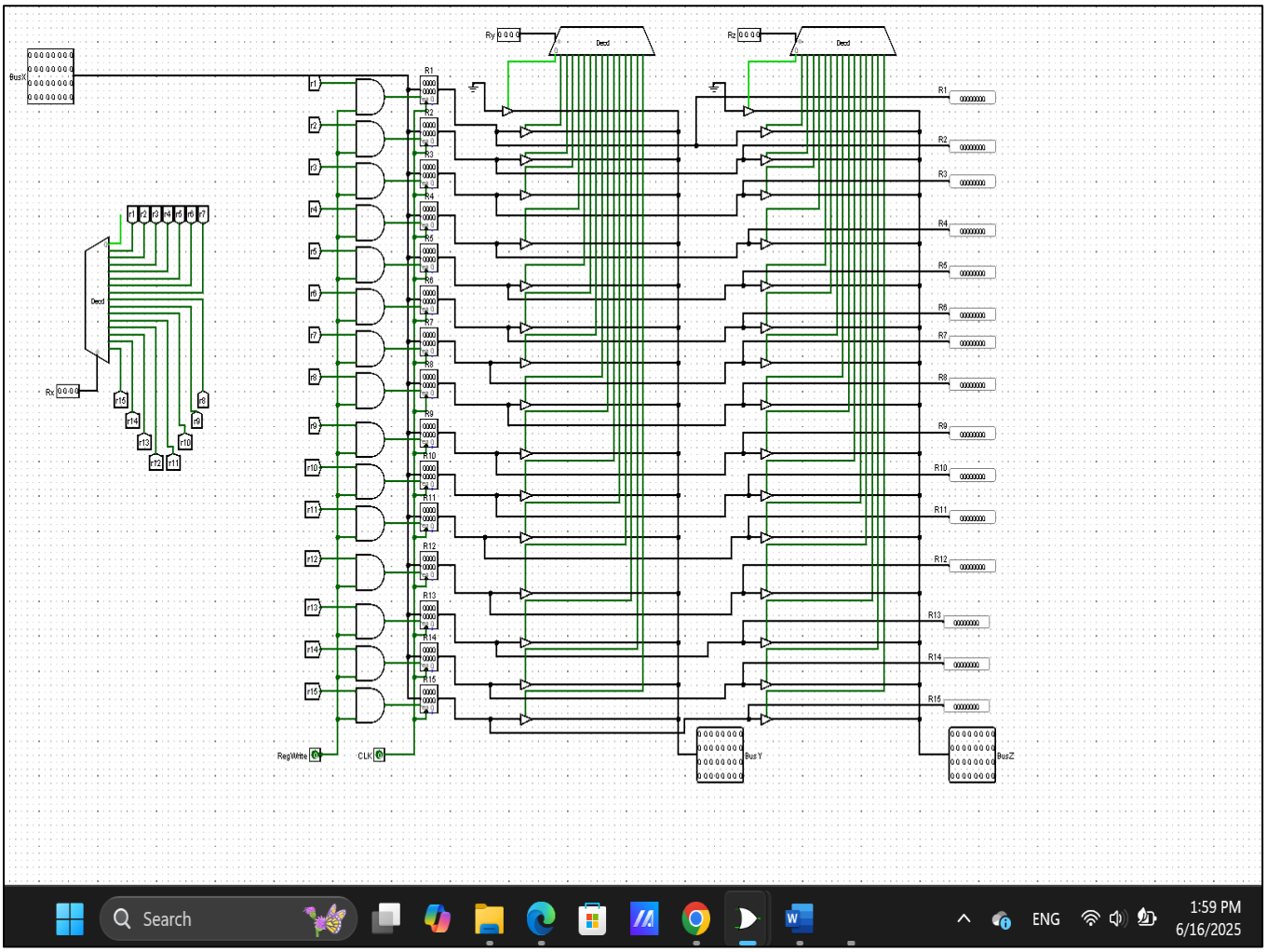


Figure 1.2:The Register File of the SCP.

The register file in figure 1.2 is a main component in our single-cycle processor, as it provides temporary, high-speed storage that is faster to access than memory, enhancing the performance of the processor.

Our design is made up of sixteen 32-bit registers, labeled R0 to R15, where R0 is a special grounded register, so that reading R0 always returns zero value and writing R0 has no effect, it is impractical to use R0 unless we want to read a value of 0. Our register file has two read ports, labeled as BusY and BusZ, and one write port labeled as BusX, which makes it efficient to read two operands and write a result at the same time, which matches the purpose of a single-cycle processor.

At the rising edge of the clock and when RegWrite is enabled with “1”, Input Rx, which is a 4-bit input, is used to choose which register to write on so that data on BusX is written on the

specified register. Furthermore, Inputs Ry and Rz are also 4-bit inputs that are used to specify which registers we want to read from, so that data is copied from registers to BusY or BusZ.

As seen in figure 1.2 our register file is made up of 3 4-16 decoders used to decode one of the sixteen registers, as well as fifteen AND gates, where each of them is connected to the enable of a register, and all registers are connected to a main clock. The output of each register is connected to the input of a tristate buffer, and each output of the decoder is connected to an enable of a tristate buffer. When the enable is high, the tristate buffer is connected, but when the enable is low, the tristate buffer is an open circuit (high impedance state). Therefore, each of BusY and BusZ will read its data from one register at each clock cycle.

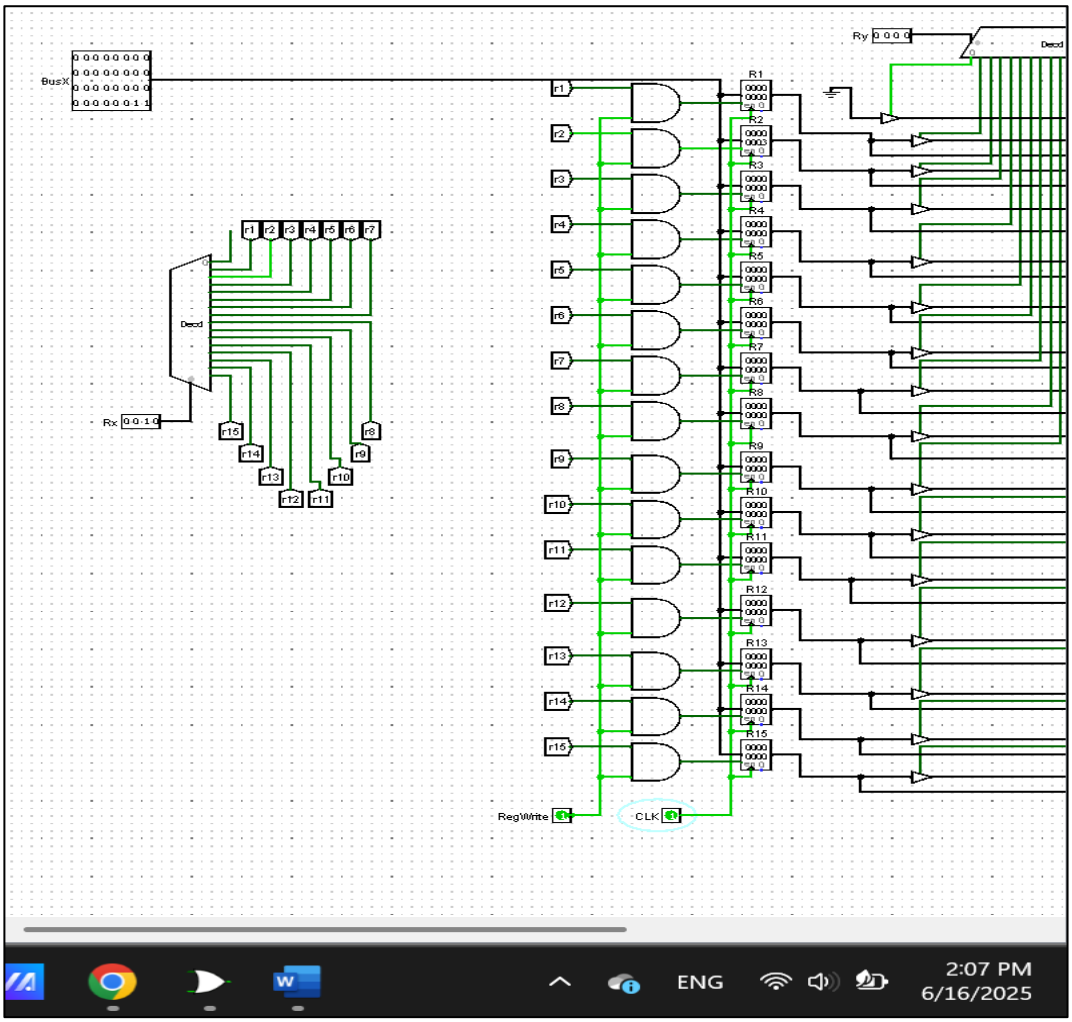


Figure 1.3:Writing Operation on Register File

In Figure 1.3, a writing operation was performed by entering binary data “11” (0x3) on BusX, we also entered 0010 on Rx to choose register number 2 to write on. We also had RegWrite Signal enabled “1” and we clicked on the clock, then we noticed that 0x3 is now written on Register 2.

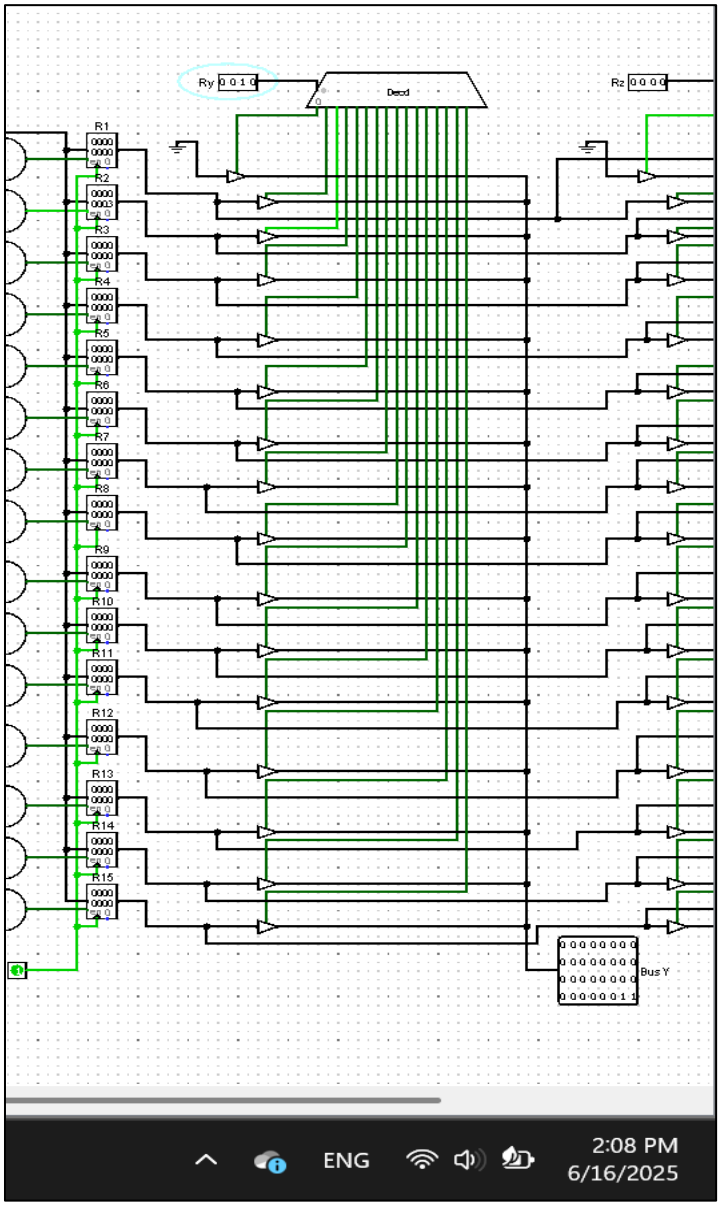


Figure 1.4:Reading Operation on Register File.

In Figure 1.4, the reading operation was performed by entering the value 0010 on Ry, which is an input to a 4-16 decoder, so that output 2 of the decoder (which is shown in light green) will be high and its tristate buffer will be connected and will pass the value at register to BusY. Now BusY clearly shows the data “11” on it.

1.2 ALU (Arithmetic and Logic Unit):

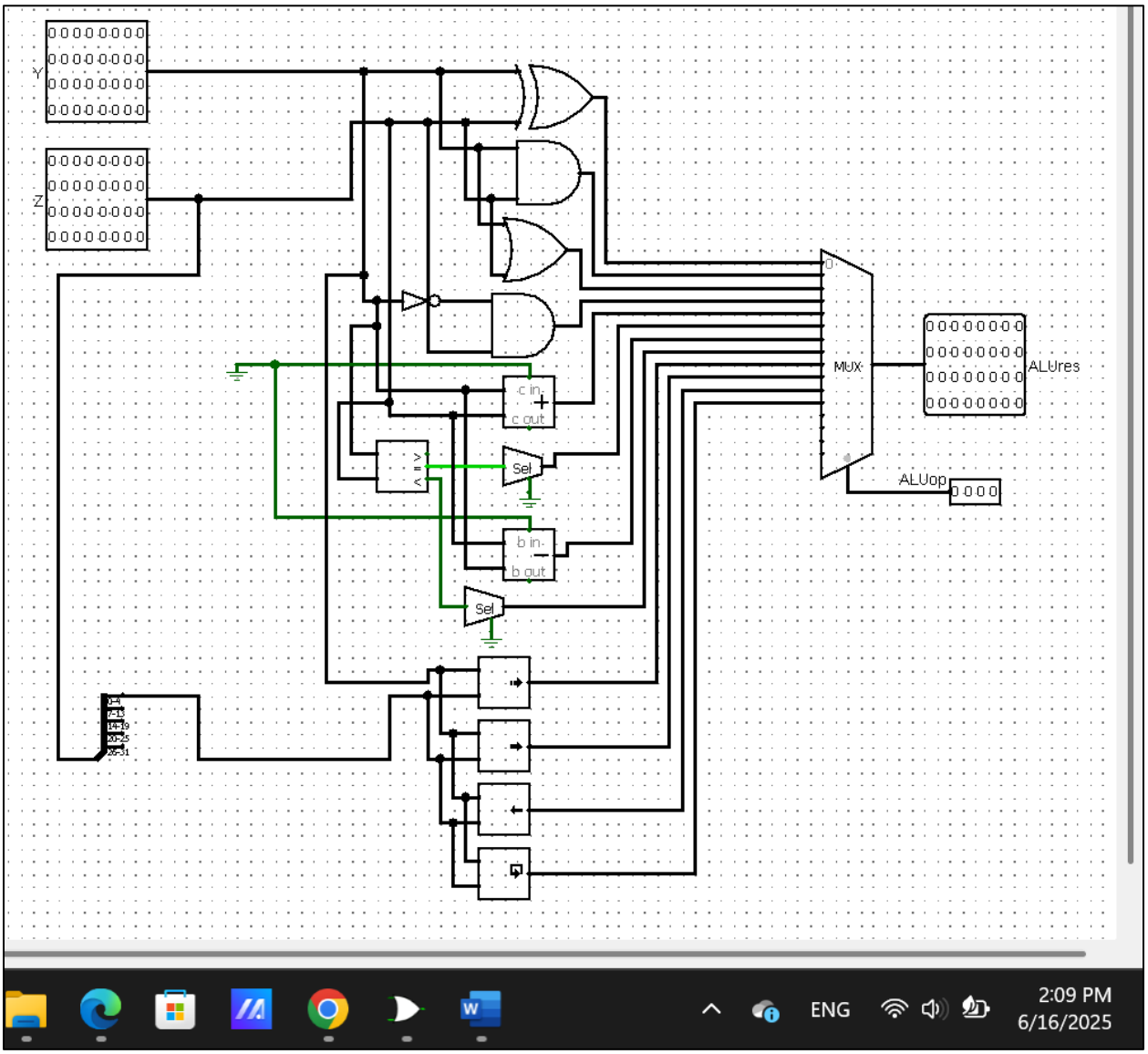


Figure 1.5:ALU File

The ALU in figure 1.5 is an important component that performs all arithmetic and logical functions in the processer. ALU file is made up of a 16-1 MUX (because we needed 11 arithmetic and logical operations) that takes an ALU opcode to allow one of the inputted operations to be executed, it also takes two 32 bit input ports Y and Z to receive two operands and has one 32-bit output port which is the ALUres. The four-bit selection lines of the MUX is labeled here as 'ALUop' is a control signal that will be generated int the main control unit.

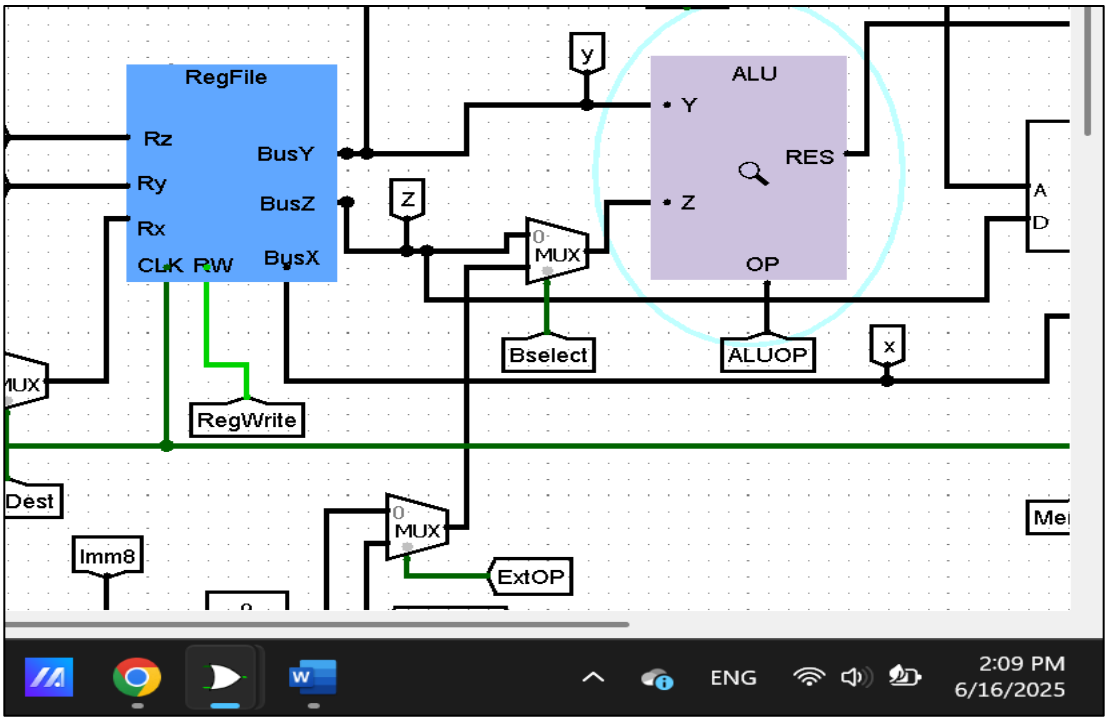


Figure 1.6: ALU block in datapath

In our datapath, ALU block, which is shown in purple in figure 1.6, receives input Y directly from BusY in register file discussed above, while ALU input Z is received from a mux that chooses between BusZ from register file or another MUX that chooses between an 8-bit immediate number that is either zero extended or signed extended.

Table 1.1 shows each ALU mux input in the ALU file

ALUop	Operation name	Description
0000	XOR	Rx=Ry^Rz
0001	AND	Rx=Ry&Rz
0010	OR	Rx=Ry|Rz
0011	CAND	Rx=~Ry&Rz
0100	ADD	Rx=Ry+Rz
0101	SEQ	Rx = (Ry == Rz) (result is 0 or 1)
0110	NADD	Rx = –Ry + Rz
0111	SLT	Rx = (Ry < Rz) (result is 0 or 1)
1000	SRA	Rx = Shift_Right_Arithmetic(Ry, Imm8)
1001	SRL	Rx = Shift_Right_Logical(Ry, Imm8)
1010	SLI	Rx = Shift_Left_Logical(Ry, Imm8)
1011	ROR	Rx = Rotate_Right(Ry, Imm8)
1100 to 1111	XXX	Don’t care

Table 1.1:ALUop description

1.3 Data Memory

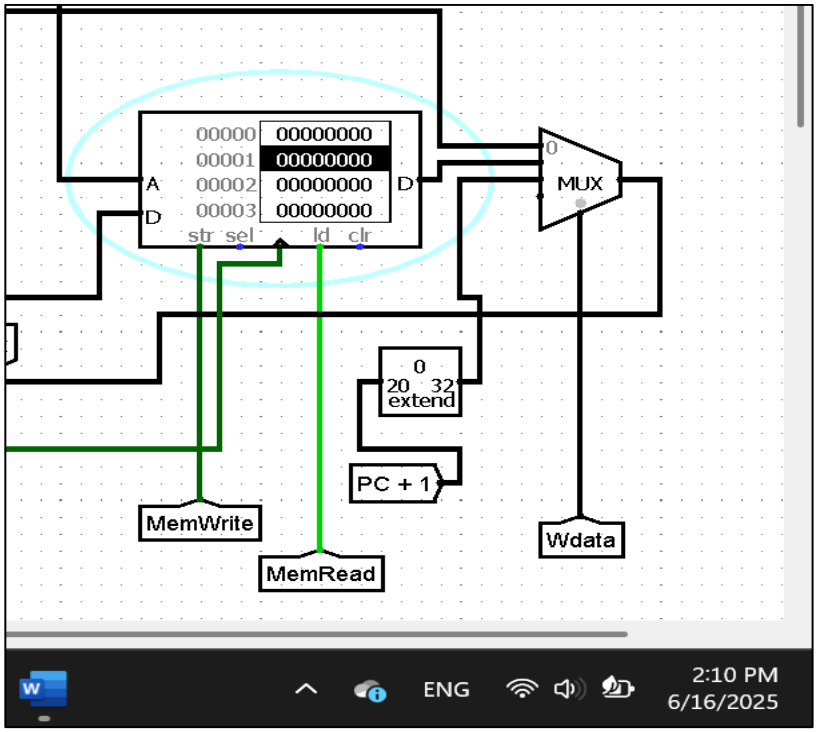


Figure 1.7:Data Memory.

The data memory in figure 1.7, which is circled in blue, is the part of the single cycle processor where our data is stored. When an instruction is executed, the data memory can either accept data to store it in a specified address (STORE instruction) or it can load data from a specified address to the register file (LOAD instruction).

In figure 1.7, our data memory is RAM memory where the data bit width is 32 bits, and the address bit width is 20 bits, so that the size of the memory is 4MB. The RAM has different input ports, data 32-bit input port, address 20-bit port, clock, clear, read enable and write enable. It also has one 32-bit output port.

In our design, the RAM receives data from BusZ in register file, it also receives address of BusY added to Imm8, which matches the load instruction provided in our project description. The RAM output is connected to a mux so that it is selected in some cases when we need to read from memory and send it to the register file.

1.4 Instruction memory

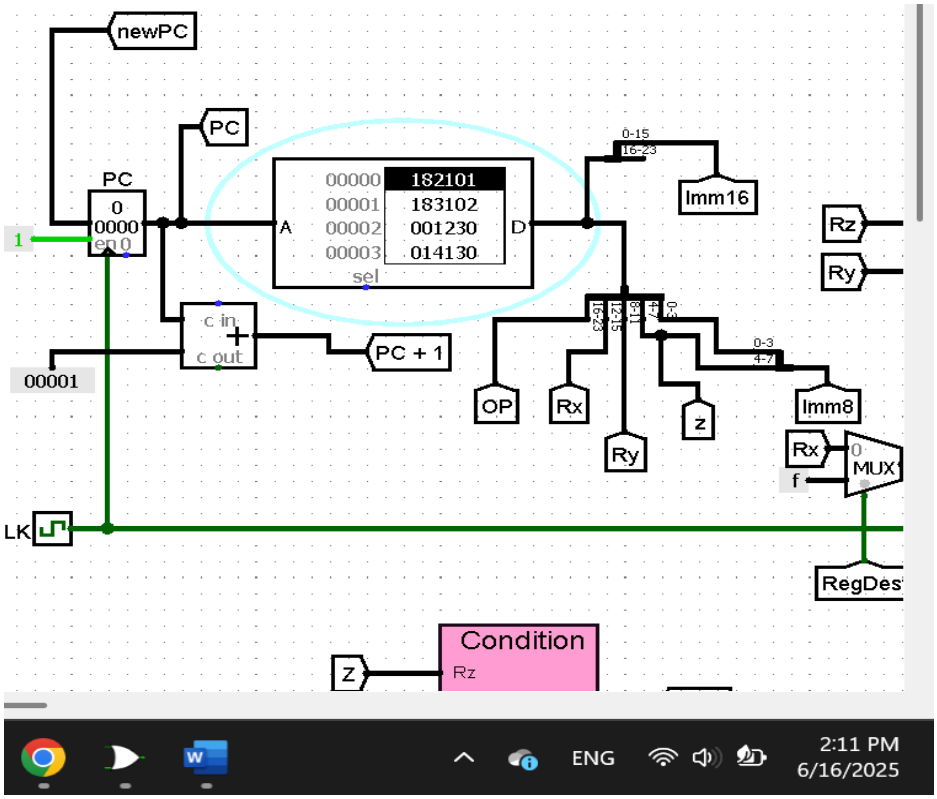


Figure 1.8:Instruction memory.

Figure 1.8 shows our instruction memory, which is where all our program instructions are stored. It is basically a ROM memory that receives the address of the next instruction to be executed which is stored in the PC register, however the newPC will be discussed in next sections. It then locates the instruction in the ROM and splits it into different parts.

If the stored instruction is R-type, then the least 4 bits are not used, bits 4-7 represent Rz register, bits 8-11 represent Ry register, bits 12-15 represent Rx register, and bits 16-23 are the opcode.

If the stored instruction is I-type, then the least 8 bits are an 8-bit immediate, bits 8-11 represent Ry register, bits 12-15 represent Rx register, and bits 16-23 are the opcode.

In case of J-type instruction, then bits 0-15 are a 16-bit immediate and bits 16-23 are the opcode.

Each of the outputs of the splitter will be sent to appropriate destinations in the processer to continue execution of the program.

1.5 PC control

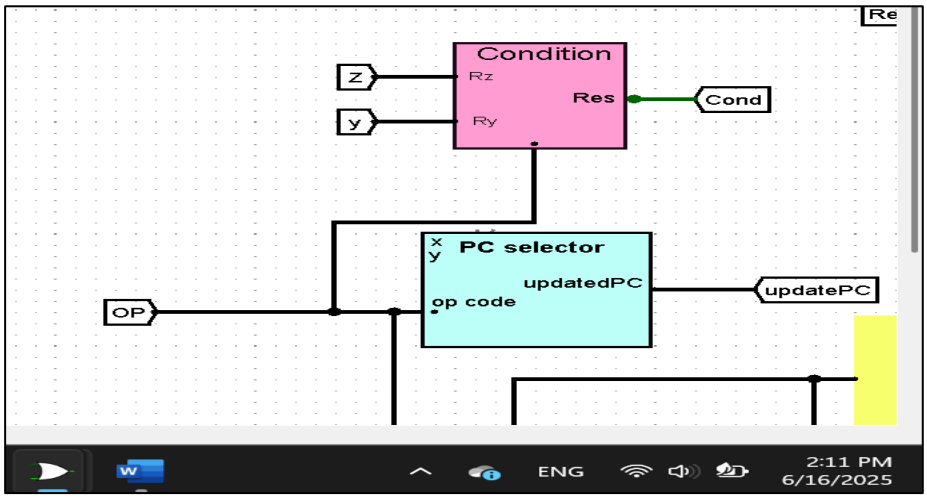


Figure 1.9:PC controllers in datapath

1.5.1 Condition Unit

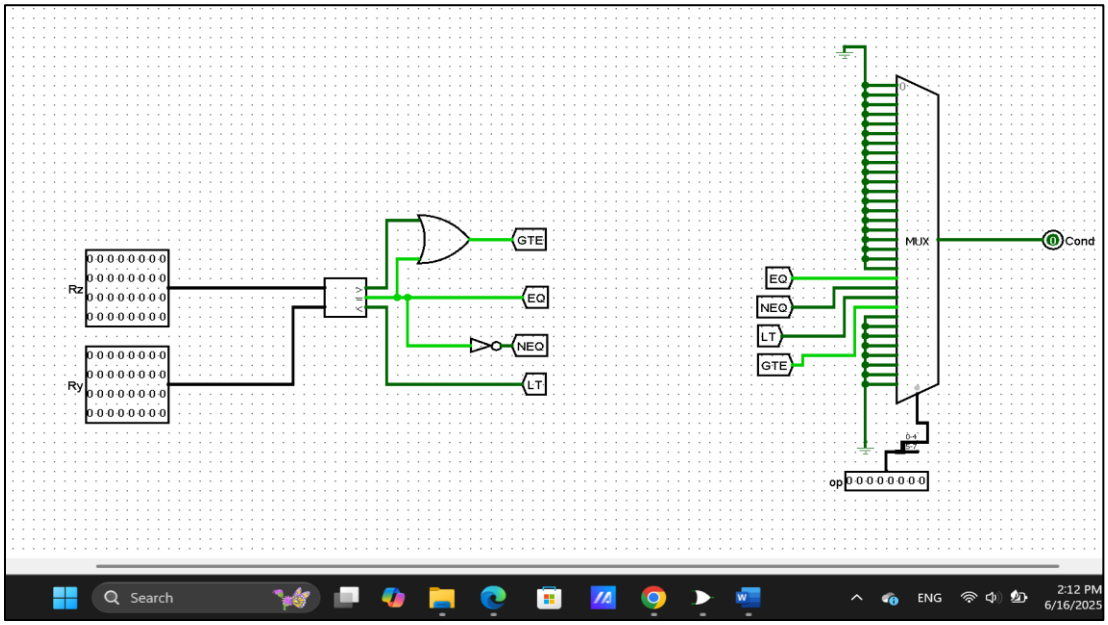


Figure 1.10:PC Condition Unit

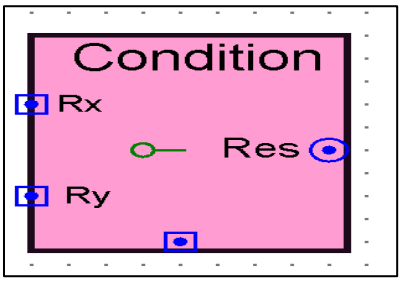


Figure 1.11:PC Condition block

In figure 1.10, the comparator unit on the left receives two 32-bit inputs Rx and Ry and then compares their values by deciding if Rx is greater than or equal to Ry, if Rx equals Ry, if Rx does not equal Ry, and if Rx is less than Ry.

The mux on the right is a 32 to 1 mux because the select bits are the 5 least bits of the opcode, and the inputs of the MUX are either an output of the comparator placed at a specified mux input based on the instruction, otherwise the input is grounded. For example, if the instruction is BEQ (opcode =20) then the output of the mux (which is called cond) will take the "EQ" value that came out of the comparator.

The output "Cond" will be sent to a mux in section 1.6 to update PC.

1.5.2 PC Selector

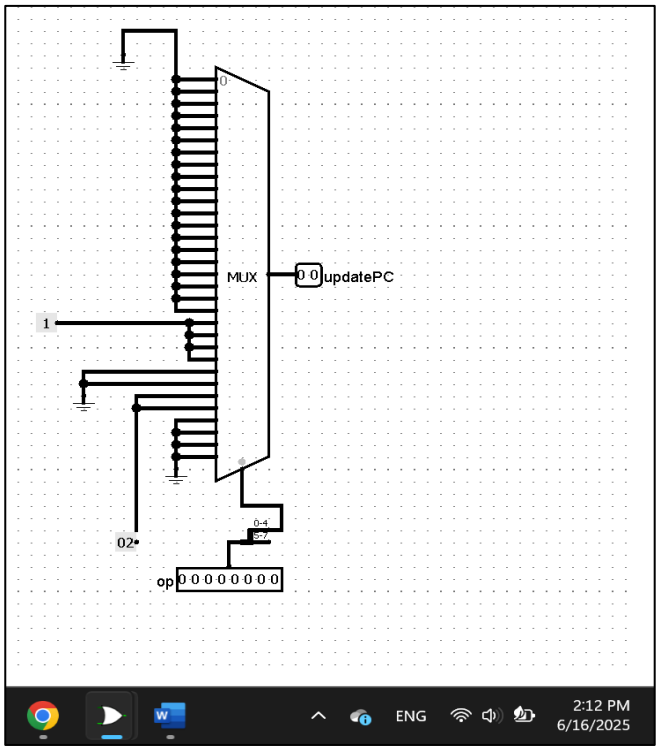


Figure 1.12:PC selector

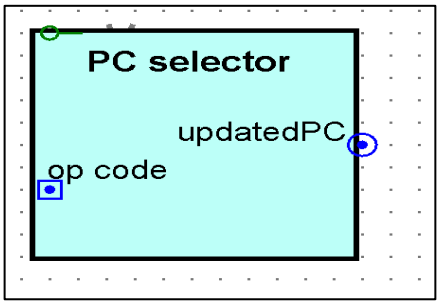


Figure 1.13: PC selector block

Figure 1.12 shows PC selector unit which is made up of a 32 to 1 mux as it takes the least 5 bits of the opcode, it then generates a 2-bit output signal called updatePC. The updatePC signal is connected as mux selector in section 1.6.

Instruction type	updatePC
R-type, I-type(except Branch instructions)	00
Branch instructions	01
J-type	10

1.6 Controlling PC in Datapath

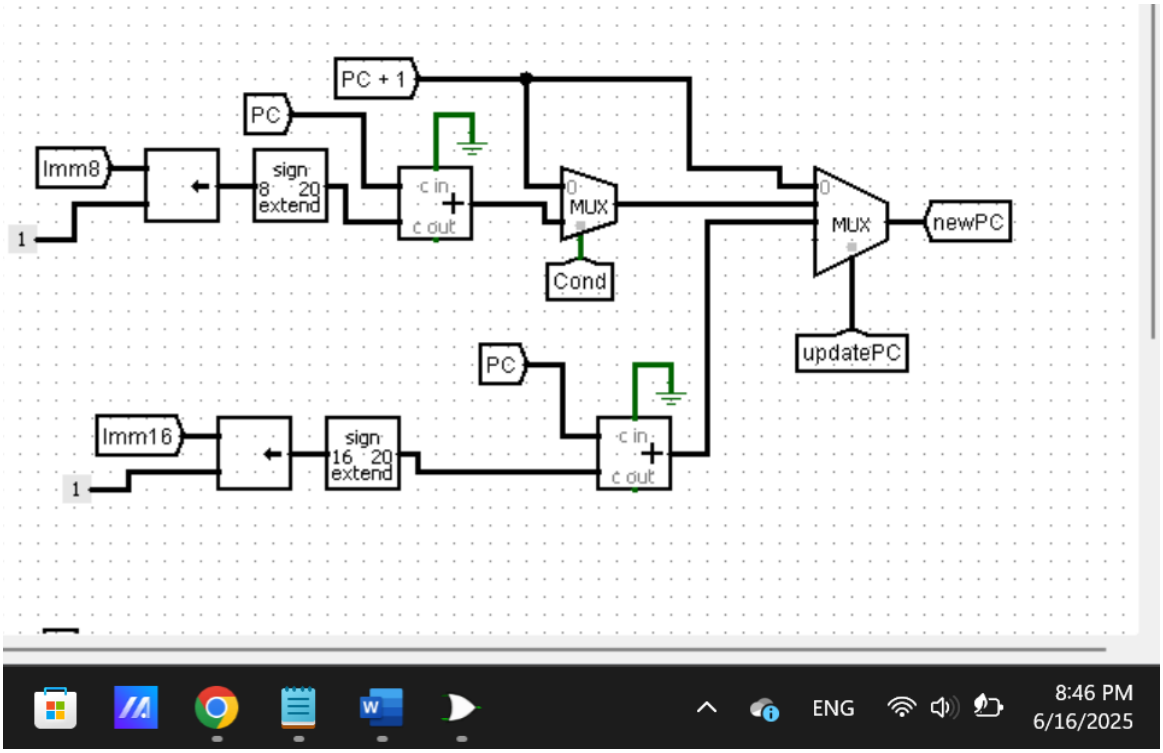


Figure 1.14:PC controller in Datapath

As shown in figure 1.14, the 4 to 1 mux on the right receives the updatePC that was discussed in section 1.5.2, it is basically a selection line that decides the newPC based on the instruction type. The newPC will be sent to the instruction memory.

On input 00 of the mux, the new PC will be PC+1, which means that the processor will move to the next instruction if the previous was of R-type and I-type (except Branch instructions).

On input 01, we notice a 2 to 1 mux that has the signal “cond”, which was discussed in section 1.5.1, as its selection line. When Cond is 0, it means that the branching condition was not satisfied, and the processor moves to the next instruction. when Cond is 1, the new Pc takes an address

resulted from adding the original PC to a logical shifted left 8-bit immediate number by one bit and then sign extended to become 20 bits, which means that the branch condition was satisfied.

Furthermore, on input 10 of the updatePC signal on mux, the new PC will result from adding the original PC to logical left shifted 16-bit immediate bit by 1-bit and then sign extended to become 20 bits. This case represents the jump instructions which are unconditional branching.

1.7 The Main Control Unit

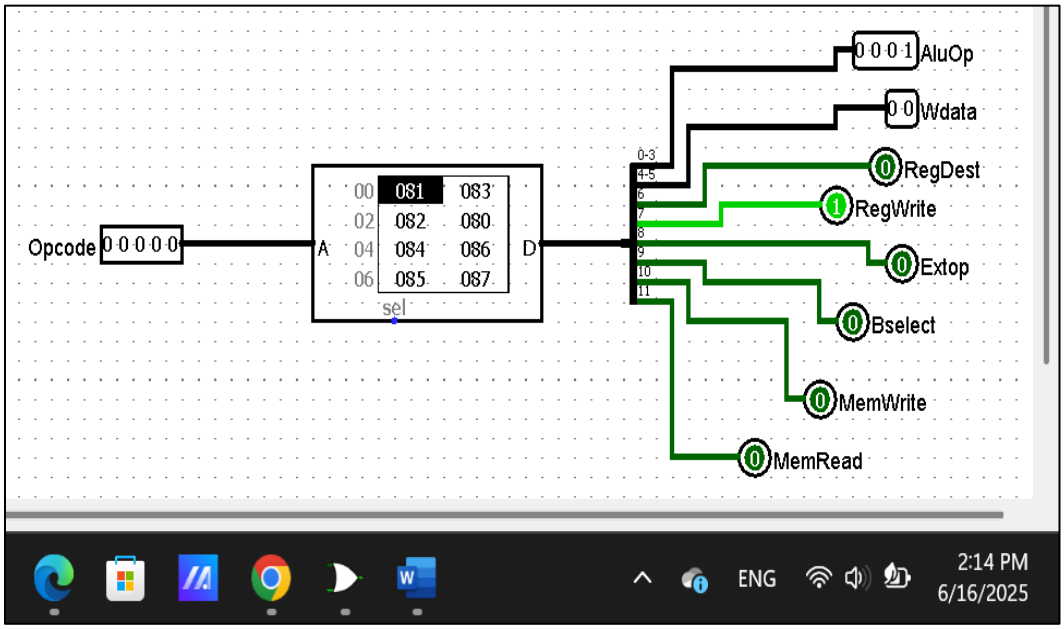


Figure 1.15: The Control Unit

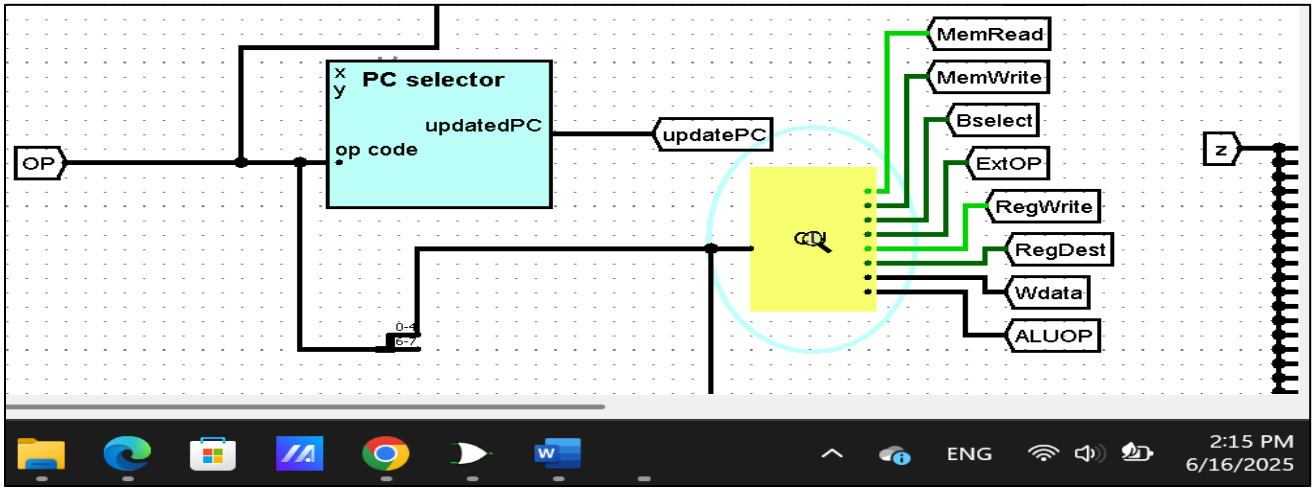


Figure 1.16: Control Unit in Datapath

Figure 1.15 shows the control unit which is responsible for sending control signals to different parts of the processor to control the execution of the instructions. It takes the instruction's Opcode and uses a ROM memory which is a microprogrammed unit to generate signals that enable or disable other components.

Table 1.2 describes the control signals which are outputs of the control unit:

Control Signal	Number of bits	Description	Signals meaning
MemRead	1	Enables reading from memory (for load instruction)	0->no read 1->read
MemWrite	1	Enables writing on memory (for store instruction)	0->no write 1->write
Bselect	1	To determine the second operand of the ALU	0->BusZ 1->extended Imm8
ExtOP	1	To determine whether to extend imm8 by zeros or by sign	0->imm8 zero extended 1->imm8 sign extended
RegWrite	1	Enables writing on register file	0->no write 1->write
RegDest	1	To determine the destination register	0->Rx 1->R15
Wdata	2	To determine what data to be written back on register file	00->ALU result 01->Data read from memory 10->PC+1 11->XX
AluOp	4	To determine the ALU operation	Shown in Table 1.1

Table 1.2:Control Unit Output signals

Table 1.3 shows how each instruction is decoded and the signal generated.

	MemRead	MemWrite	Bselect	ExtOP	RegWrite	RegDest	Wdata	AluOp	Hex value
AND	0	0	0	X(0)	1	0	00	0001	0x081
CAND	0	0	0	X(0)	1	0	00	0011	0x083
OR	0	0	0	X(0)	1	0	00	0010	0x082
XOR	0	0	0	X(0)	1	0	00	0000	0x080
ADD	0	0	0	X(0)	1	0	00	0100	0x084
NADD	0	0	0	X(0)	1	0	00	0110	0x086
SEQ	0	0	0	X(0)	1	0	00	0101	0x085
SLT	0	0	0	X(0)	1	0	00	0111	0x087

ANDI	0	0	1	1	1	0	00	0001	0x381
CANDI	0	0	1	1	1	0	00	0011	0x383
ORI	0	0	1	1	1	0	00	0010	0x382
XORI	0	0	1	1	1	0	00	0000	0x380
ADDI	0	0	1	1	1	0	00	1000	0x388
NADDI	0	0	1	1	1	0	00	0110	0x386
SEQI	0	0	1	1	1	0	00	0101	0x385
SLTI	0	0	1	1	1	0	00	0111	0x387
SLL	0	0	1	1	1	0	00	1010	0x38A
SRL	0	0	1	1	1	0	00	1001	0x389
SRA	0	0	1	1	1	0	00	1000	0x388
ROR	0	0	1	1	1	0	00	1011	0x38B
BEQ	0	0	X(0)	X(0)	0	X(0)	01	X(1111)	0x01F
BNE	0	0	X(0)	X(0)	0	X(0)	01	X(1111)	0x01F
BLT	0	0	X(0)	X(0)	0	X(0)	01	X(1111)	0x01F
BGE	0	0	X(0)	X(0)	0	X(0)	01	X(1111)	0x01F
LW	1	0	X(0)	X(0)	1	0	01	X(1111)	0x89F
SW	0	1	X(0)	X(0)	0	X(0)	01	X(1111)	0x51F
J	0	0	X(0)	X(0)	0	X(0)	X(11)	X(1111)	0x03F
JAL	0	0	X(0)	X(0)	1	1	10	X(1111)	0x0EF

Table 1.3: Instructions Control Signal Encoding

2. Simulation and Testing

2.1 Instruction Set 1: (R-type instructions)

Table 2.1 shows the instructions that we wrote to test R-type instructions sequentially.

#	Address of instruction	Assembly	Binary	Hex
1	0x00000	LW R2,R1,#0x01	00011000/0010/0001/00000001	0x182101
2	0x00001	LW R3,R1,#0x02	00011000/0011/0001/00000010	0x183102
3	0x00002	AND R1,R2, R3	00000000/0001/0010/0011/0000	0x001230
4	0x00003	CAND R4, R1, R3	00000001/0100/0001/0011/0000	0x014130
5	0x00004	OR R4, R1, R3	00000010/0100/0001/0011/0000	0x024130
6	0x00005	XOR R5, R1, R3	00000011/0101/0001/0011/0000	0x035130
7	0x00006	ADD R5,R4,R2	00000100/0101/0100/0010/0000	0x045420
8	0x00007	NADD R4, R6, R5	00000101/0100/0110/0101/0000	0x054650
9	0x00008	SEQ R4, R1, R2	00000110/0100/0001/0010/0000	0x064120
10	0x00009	SLT R4, R1, R6	00000111/0100/0001/0110/0000	0x074160

Table 2.1: Instruction set 1

Data initially in RAM:

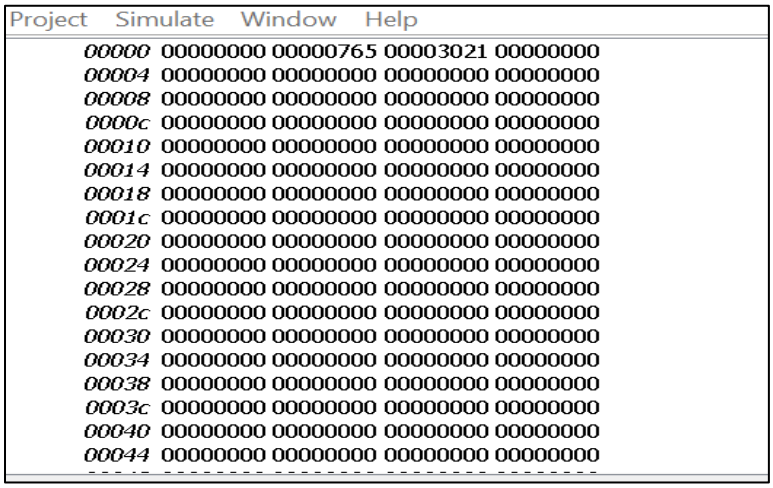


Figure 2.1:initial data in RAM

Instructions initially in ROM:

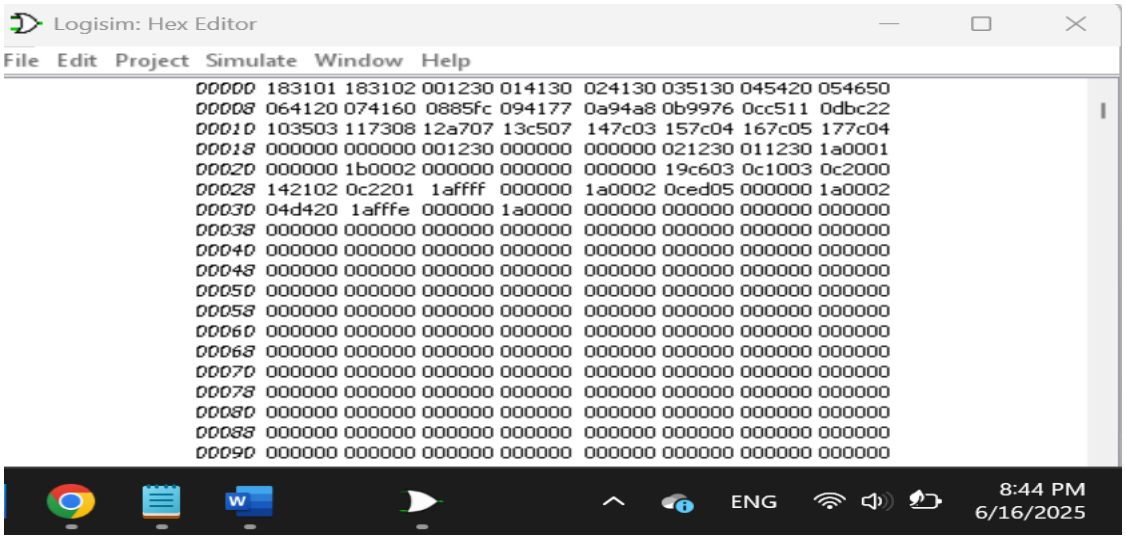


Figure 2.2: initial instructions in ROM

The initial Register File (Before Any Instructions):

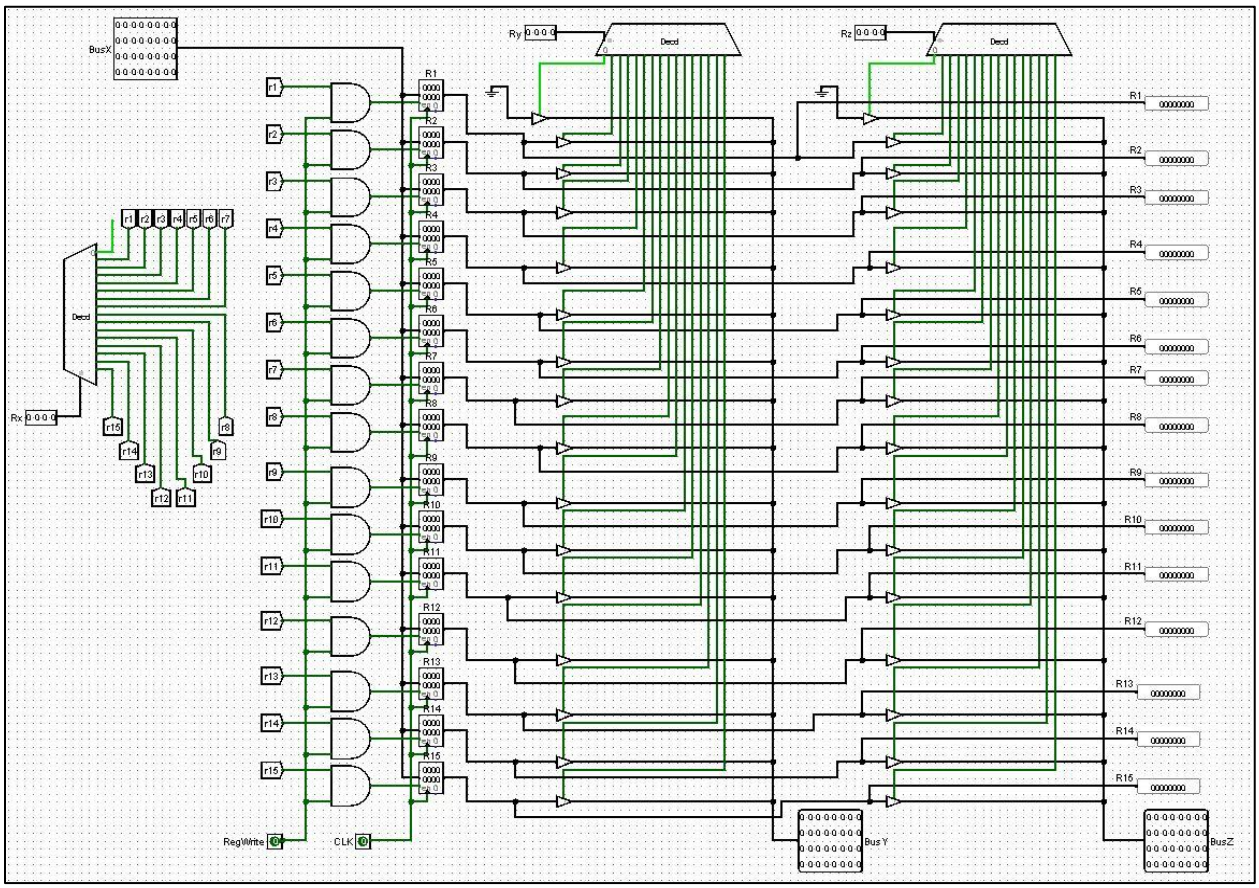


Figure 2.3: initial register file

2.1.1 Simulating the First Two instructions:

First instruction is LW R2, R1, #0x01 which means load contents of memory address R1+0x01 to R2.

R2=M[R1+0x00000001] =Mem[0x00000001] = 0x00000765, so R2 now have 0x00000765.

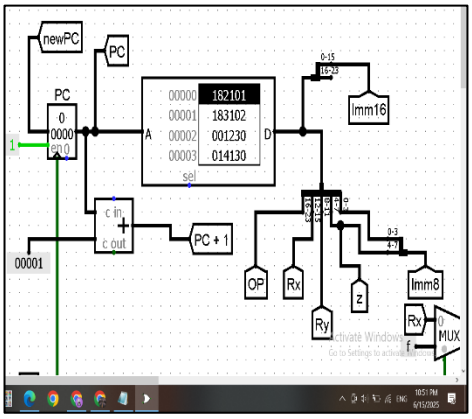


Figure 2.4: First Instruction in Instruction Memory.

Second Instruction is LW R3, R1, #0x02 which means load contents of memory address R1+0x02 to R3.

R3 = Mem[R1+ imm8] = Mem[0x0002] =0x00003021, so R2 now have 0x00003021 as shown in figure 2.6.

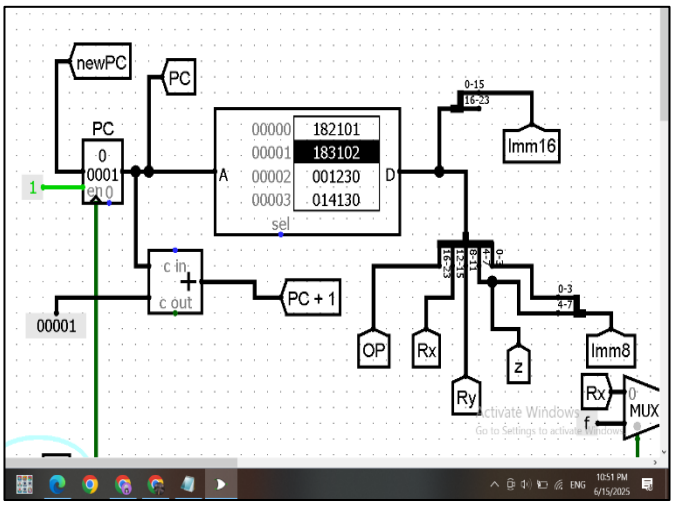


Figure 2.5: Second Instruction in Instruction Memory.

Figure 2.6 shows the contents of R2 and R3 after simulating the First Two instructions:

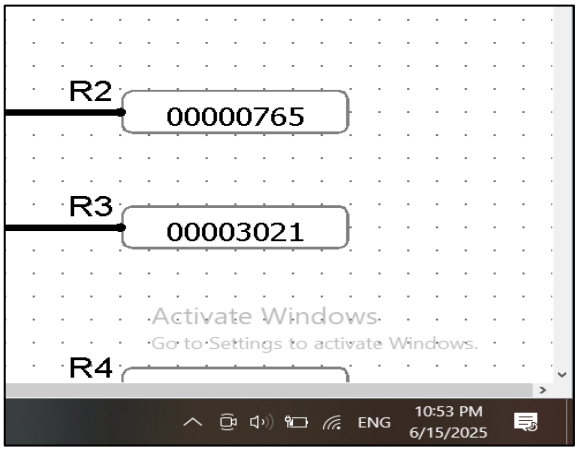


Figure 2.6: Result of Load Operation to R1 and R2

2.1.2 Simulate the Third instruction:

In figure 2.7, the third instruction is AND R1, R2, R3 which means save result of AND operation between R2 that contain 0x00000765 and R3 that contain 0x00003021 in R1.

R1= R2 & R3 = 0x00000021. So R1 now have 0x00000021 as shown in figure 2.8.

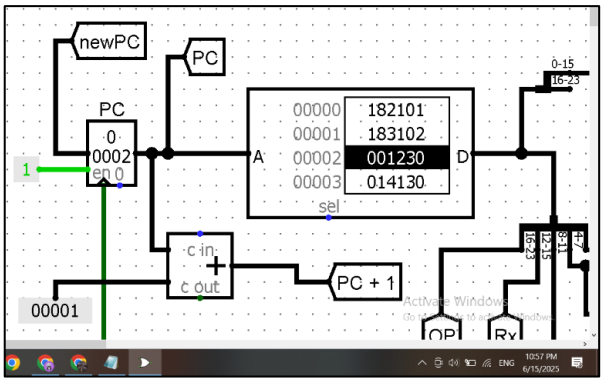


Figure 2.7: Third Instruction in Instruction Memory.

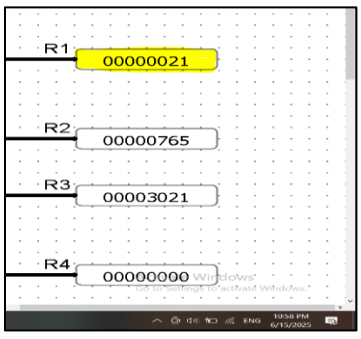


Figure 2.8: Result of R1 after AND operation

2.1.3 Simulating the Fourth instruction:

In figure 2.9, the fourth instruction is CAND R4, R1, R3 which means save the result of CAND operation between R1 that contain 0x00000021 and R3 that contain 0x00003021 in R4.

R4= ~R1 & R3 = 0x00003000. So R4 now have 0x00003000 as shown in figure 2.10.

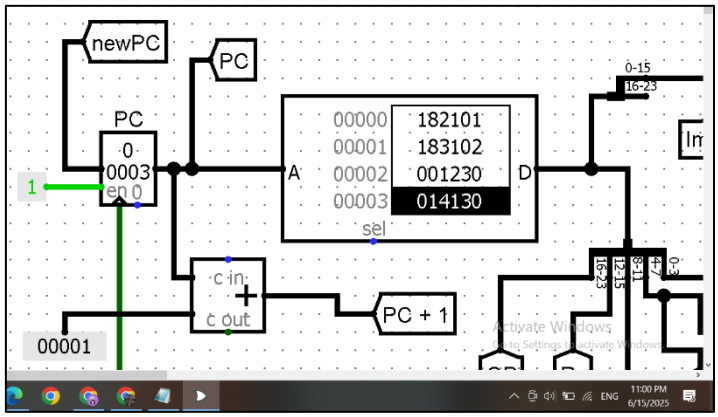


Figure 2.9: Fourth Instruction in Instruction Memory.

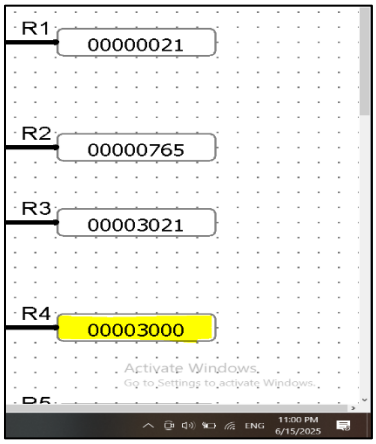


Figure 2.10: R4 after CAND operation

2.1.4 Simulating the Fifth instruction:

In figure 2.11, the fifth instruction is OR R4, R1, R3 which means save the result of OR operation between R1 that contain 0x00000021 and R3 that contain 0x00003021 in R4.

R4= R1 | R3 = 0x00003021. So R4 now have 0x00003021 as shown in figure 2.12.

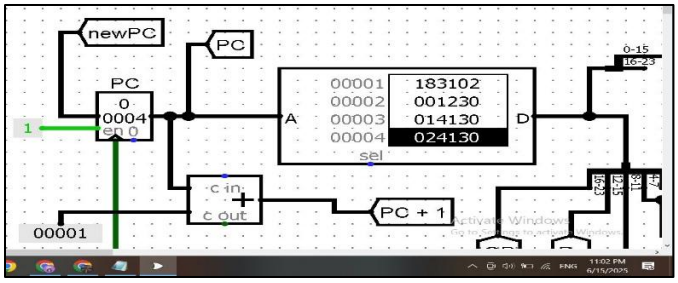


Figure 2.11: Fifth Instruction in Instruction Memory.

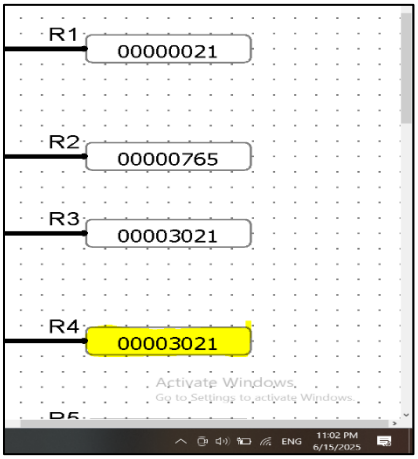


Figure 2.12: R4 after OR operation

2.1.5 Simulating the Sixth instruction:

In figure 2.13, the sixth instruction isXOR R5, R1, R3 which means save result of XOR operation between R1 that contain 0x00000021 and R3 that contain 0x00003021 in R5.

R5 = R1 ^ R3 = 0x00003000. So R5 now it has 0x00003000 as shown in figure 2.14.

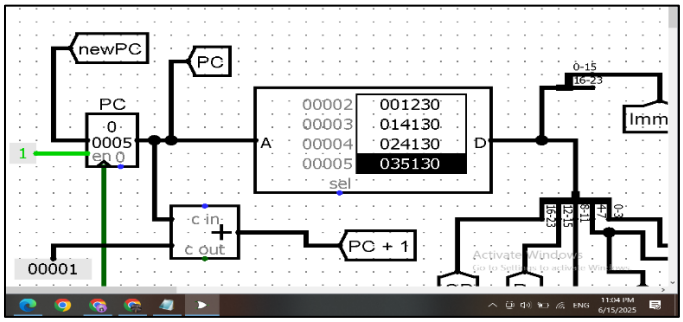


Figure 2.13: Sixth Instruction in Instruction Memory.

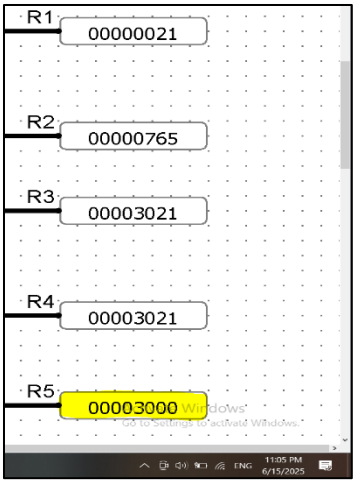


Figure 2.14: R5 after XOR Operation

2.1.6 Simulating the Seventh instruction:

In figure 2.15, the seventh instruction is ADD R5, R4, R2 which saves result of ADD (addition) operation between R4 that contain 0x00003021 and R2 that contain 0x00000765 in R5.

R5 = R4 + R2 =0x00003786. So R5 now have 0x00003786 as shown in figure 2.16.

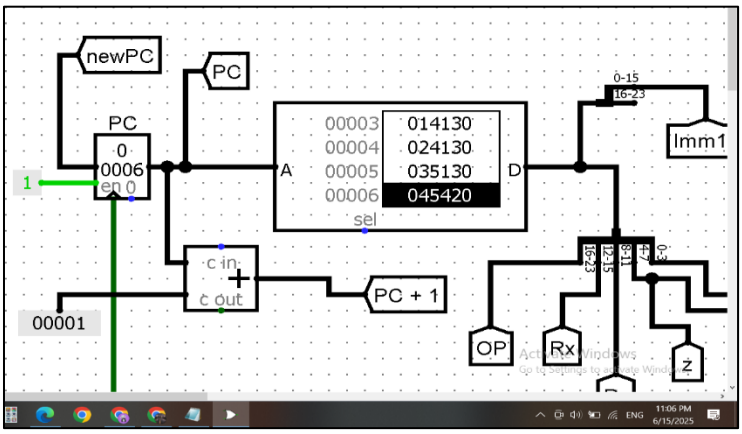


Figure 2.15: Seventh Instruction in Instruction Memory.

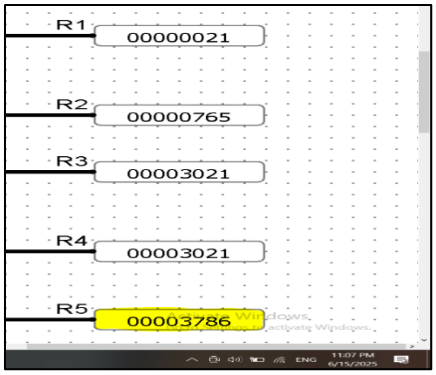


Figure 2.16: R5 after ADD operation

2.1.7 Simulating the Eighth instruction:

In figure 2.17, the eighth instruction is NADD R4, R6, R5 which means save result of NADD operation between R6 that contain 0x00000000 (It's the initial value because we did not assign it a new value) and R5 that contain 0x00003786 in R4.

R4 = -R6 + R5=0x00003786. So R4 now have 0x00003786 as shown in figure 2.18.

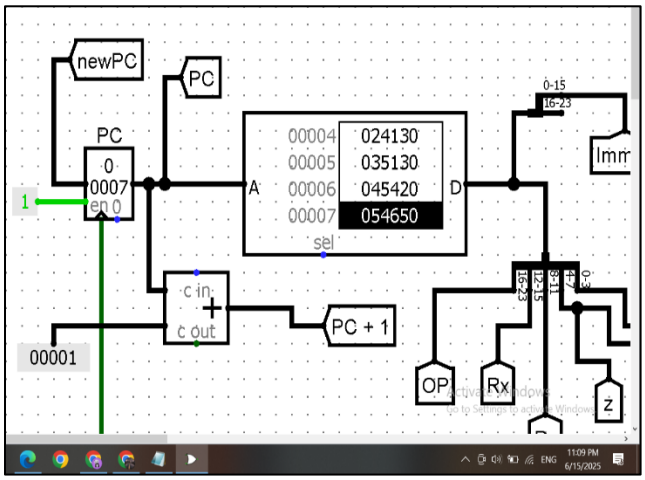


Figure 2.17: Eighth Instruction in Instruction Memory.

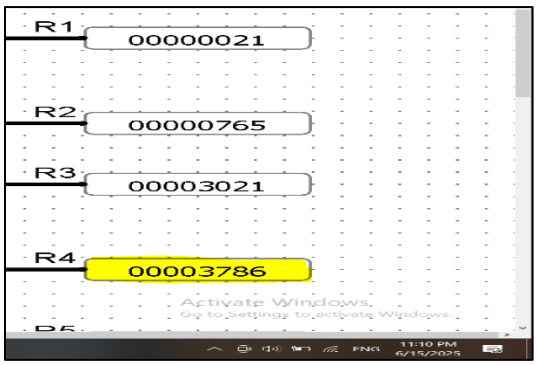


Figure 2.18:R4 after NADD Operation

2.1.8 Simulating the Ninth instruction:

In figure 2.19, the ninth instruction is SEQ R4, R1, R2 → save result of SEQ operation between R1 that contain 0x00000021 and R2 that contain 0x00000765 in R4.

R4 = (R1==R2) = 0x00000000. So R4 now have 0x00000000 as shown in figure 2.20.

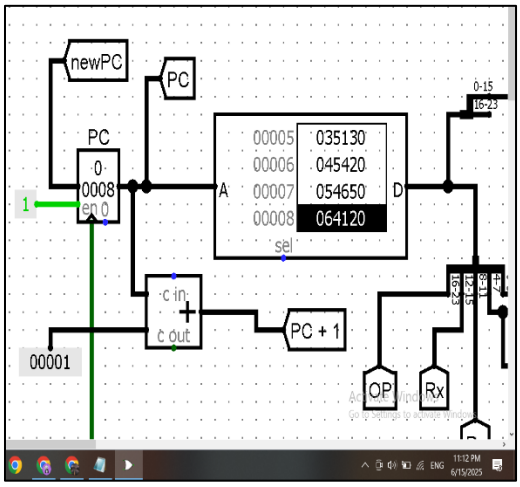


Figure 2.19: Ninth Instruction in Instruction Memory.

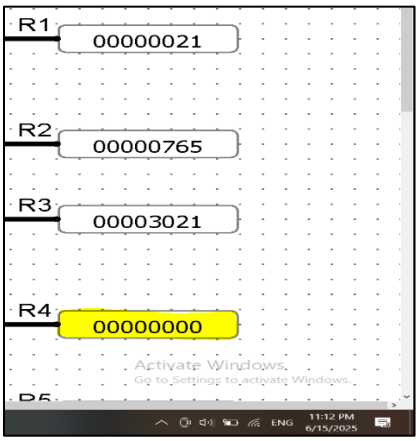


Figure 2.20:R4 after SEQ Operation

2.1.9 Simulate the tenth instruction:

In figure 2.21, the tenth instruction is SLT R4, R1, R6 which means save result of SLT operation between R1 that contain 0x0000021 and R6 that contain 0x00000000 (It's the initial value because we did not assign it a new value) in R4.

R4 = (R1 < R6) = 0x00000000. So R4 now have 0x00000000 as shown in figure 2.22.

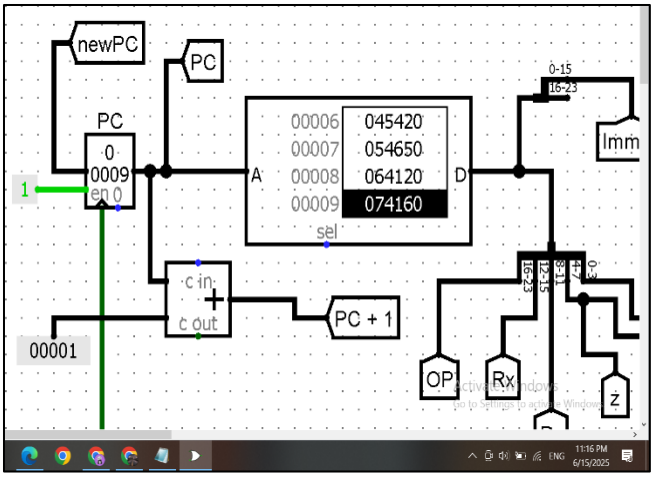


Figure 2.21: Tenth Instruction in Instruction Memory.

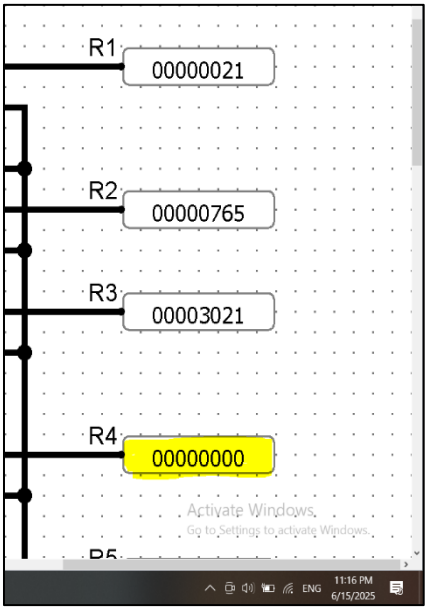


Figure 2.22: R4 after SLT Operation

2.3 Instruction Set 2: (I-type instructions)

Table 2.2 describes instruction set #2:

#	Address of instruction	Assembly	Binary	Hex
11	0x0000a	ANDI R8,R5,#0xFC	00001000/1000/0101/11111100	0x0885FC
12	0x0000b	CANDI R4,R1,#0x77	00001001/0100/0001/01110111	0x094177
13	0x0000c	ORI R9,R4,#0xA8	00001010/1001/0100/10101000	0x0A94A8
14	0x0000d	XORI R9,R9,#0x76	00001011/1001/1001/01110110	0x0B9976
15	0x0000e	ADDI R12,R5,#0x11	00001100/1100/0101/00010001	0x0CC511
16	0x0000f	NADDI R11,R12,#0x22	00001101/1011/1100/00100010	0x0DBC22

Table 2.2: Instruction Set 2

2.2.1 Simulating the eleventh instruction:

Figure 2.23 shows ANDI R8, R5, #0xFC → save result of ANDI operation between R5 that contain 0x00003786 and immediate 0xFC in R8.

R8 = R5 & #0xFC = 0x00003784. So R8 now have 0x00003784 as shown in figure 2.24.

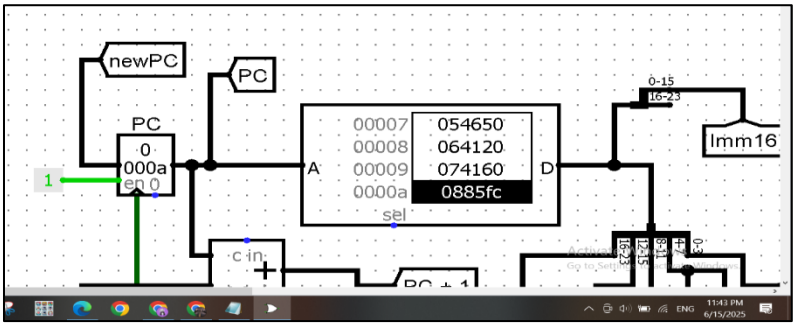


Figure 2.23:Instruction 11

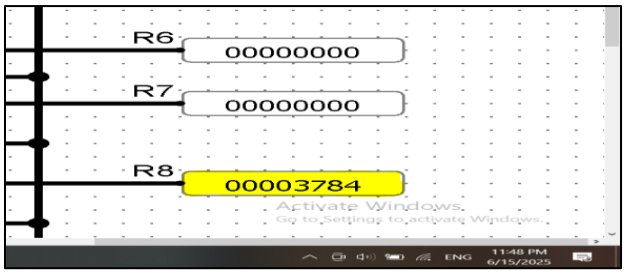


Figure 2.24:R8 after ANDI

2.2.2 Simulating the twelfth instruction:

Figure 2.25 shows CANDI R4, R1, #0x77→ save result of CANDI operation between R1 that contain 0x0000021 and immediate 0xFC in R8.

R4 = ~R1 & 0x77 = 0x0000056. So R4 now have 0x00000056 as shown in figure 2.26.

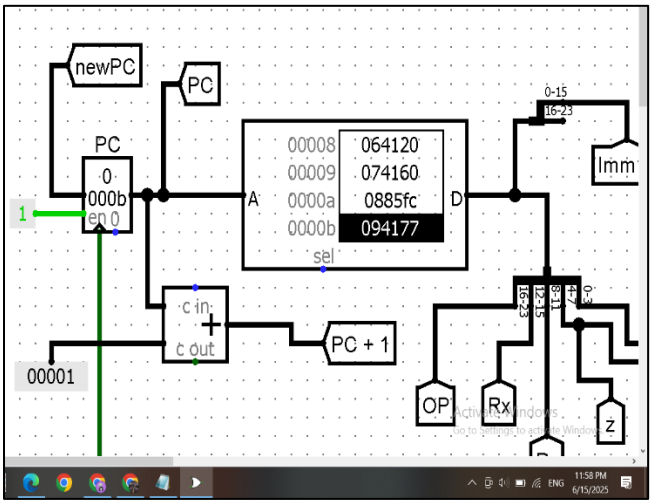


Figure 2.25:Instruction 12

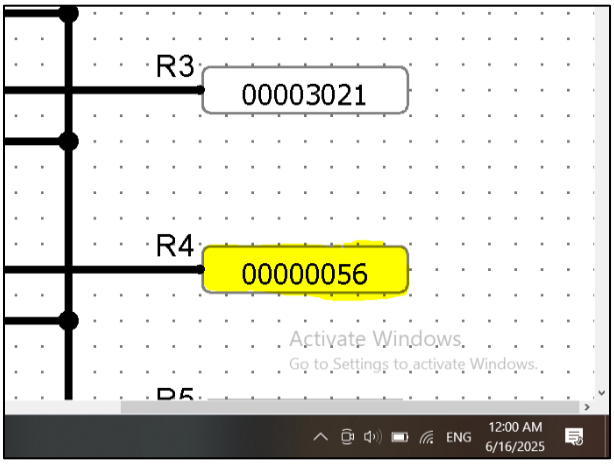


Figure 2.26: R4 after CANDI

2.2.3 Simulating the thirteenth instruction:

Figure 2.27 shows ORI R9, R4, #0xA8 → save result of ORI operation between R4 that contain 0x00000056 and immediate 0xFC in R9.

R9 = R4 | 0xA8 = 0xFFFFFFFE. So R9 now has 0xFFFFFFFE as shown in figure 2.28.

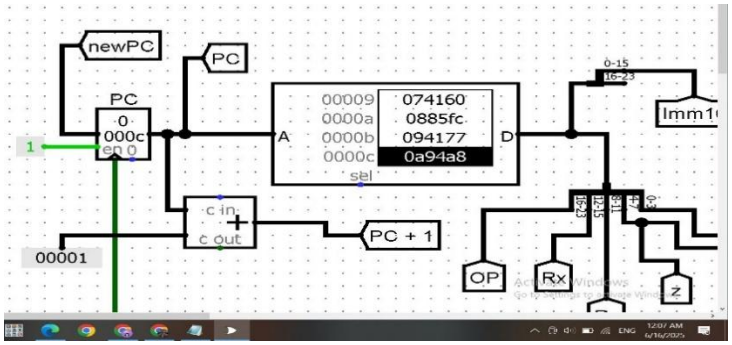


Figure 2.27 :Instruction 13 in Memory

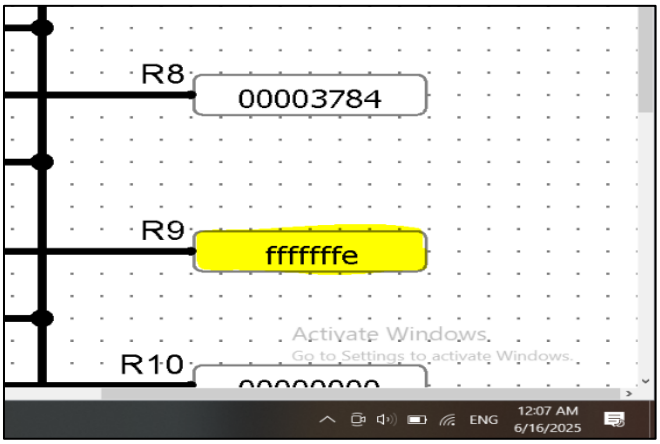


Figure 2.28:R9 after ORI

2.2.4 Simulating the fourteenth instruction:

Figure 2.29 shows XORI R9, R9, #0x76 → save result of XORI operation between R9 that contain 0xFFFFFFFE and immediate 0x76 in R9.

R9 = R9 ^ 0x76 = 0xFFFFFF88. So R9 now have 0xFFFFFF88 as shown in figure 2.30.

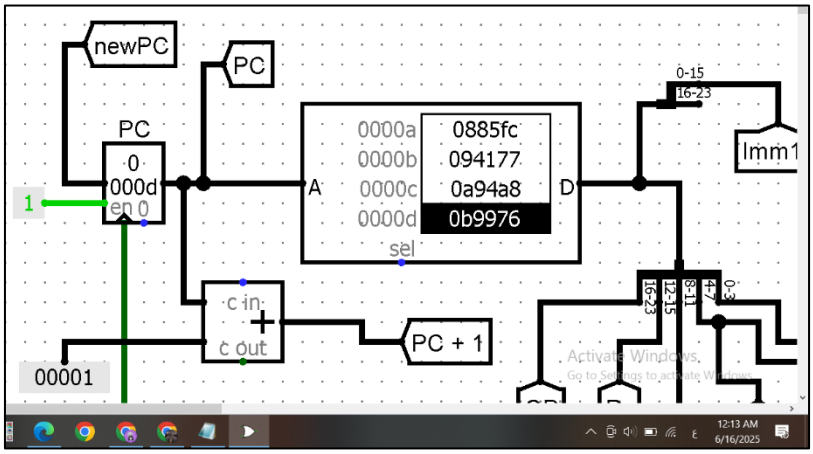


Figure 2.29:Instruction 14 in Memory

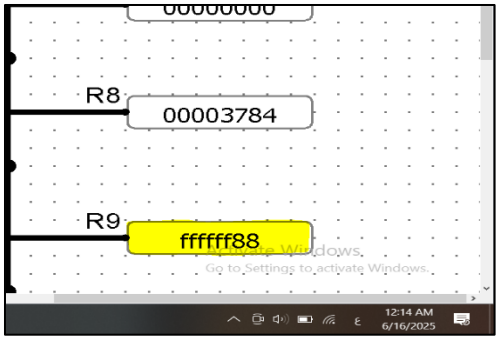


Figure 2.30:*R9 after XORI*

2.2.5 Simulating the fifteenth instruction:

Figure 2.31 shows ADDI R12, R5, #0x11 → save result of ADDI operation between R5 that contain 0x00003786 and immediate 0x11 in R12.

R12 = R5 + 0x11 = 0x00003797. So R12 now have 0x00003797 as shown in figure 2.32.

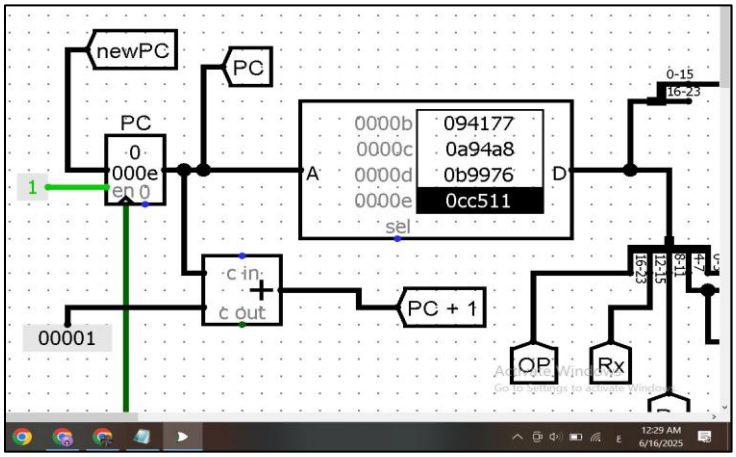


Figure 2.31:Instruction 15 in Memory

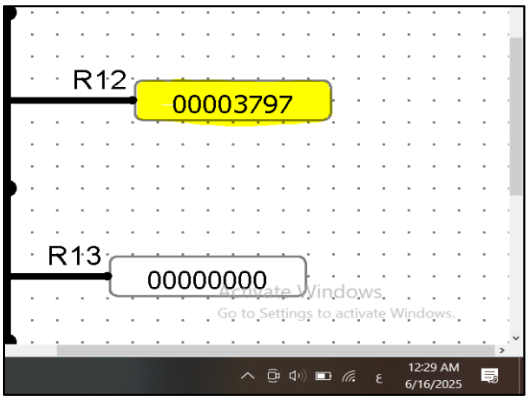


Figure 2.32: R12 after ADDI

2.2.6 Simulating the sixteenth instruction:

Figue 2.33 shows NADDI R11, R12, #0x22 → save result of NADDI operation between R12 that contain 0x00003797 and immediate 0x22 in R11.

R11 = -R12 + 0x22 = 0xFFFFC88b. So R11 now has 0xFFFFC88b as shown in figure 2.34.

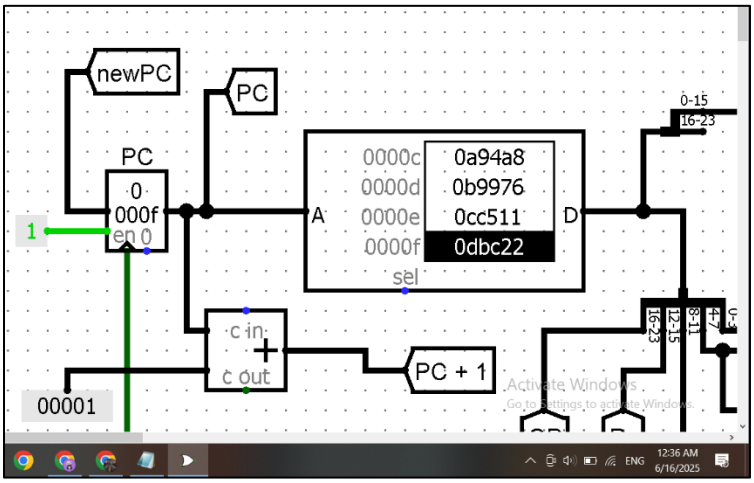


Figure 2.33: Instruction 16 in Memory

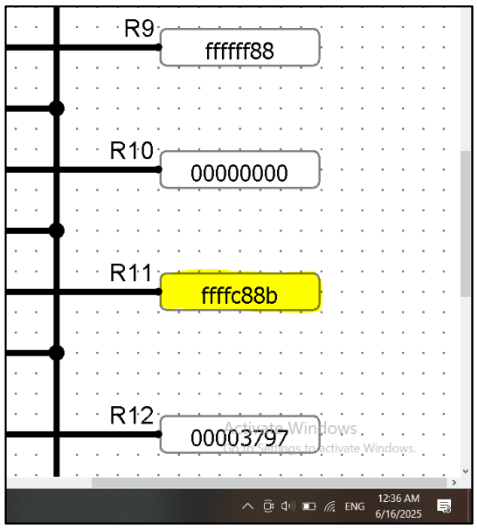


Figure 2.34:R11 after NADDI

2.3 Simulating instruction set 3

Table 2.2 shows the instructions we used to test and simulate the shift instructions, branch instructions and jump instructions.

#	Address of instruction	Assembly	Binary	Hex
17	0x00010	SLL R3, R5, #0x03	00010000/0011/0101/0000/0011	0X103503
18	0x00011	SRL R7, R3, #0x08	00010001/0111/0011/0000/1000	0X117308
19	0x00012	SRA R10, R7, #0x07	00010010/1010/0111/0000/0111	0x12A707
20	0x00013	ROR R12, R5, #0x07	00010011/1100/0101/0000/0111	0x13C507
21	0x00014	BEQ R7, R12, #0x03	00010100/0111/1100/0000/0011	0x147C03
22	0x00015	BNE R7, R12, #0x04	00010101/0111/1100/0000/0100	0x157C04
23	0x00016	BLT R7, R12, #0X05	00010110//0111/1100/0000/0101	0X167C05
24	0x00017	BGE R7, R12, #0x06	00010110/0111/1100/0000/0110	0x177C04
25	0x0001f	J 0x0001	00011010/0000/0000/0000/0001	0x1A0001
26	0x00021	JAL 0x0002	00011011/0000/0000/0000/0010	0x1B0002
27	0x0001d	OR R1, R2, R3	00000010/0001/0010/0011/0000	0x021230
28	0x00025	SW R12, R6, #0x03	00011001/1100/0110/0000/0011	0x19C603

Table 2.33: Instruction Set 3

2.3.1 Simulating the seventeenth instruction

Figure 2.35 shows instruction SLL R3, R5, #0x03 being executed. This instruction does logical left shift on the data in R5 which is 0x00003786 by 3 bits and writes the result on R3.

R3 = (R5 < 3) = 0x0001BC30 as shown in the figures below.

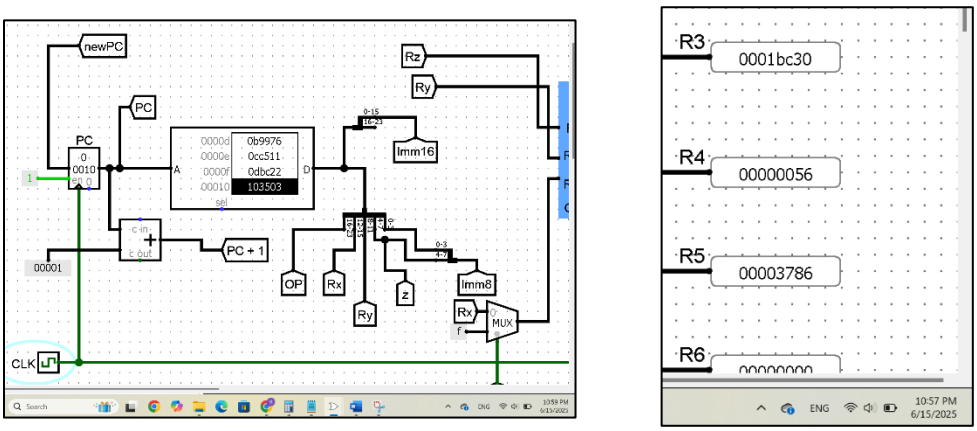


Figure 2.35: Instruction SLL Simulation

2.3.2 simulating the nineteenth instruction

Figure 2.36 shows SRA R10, R7, #0x07 instruction. This instruction does Arithmetic right shift on the data in R7 which is 0x000001BC by 7 bits and writes the result on R10.

R7 = 0x000001BC as shown in figure 2.37

R10 = (R7 > 7)

R10 = 0x00000003 as shown in figure 2.38.

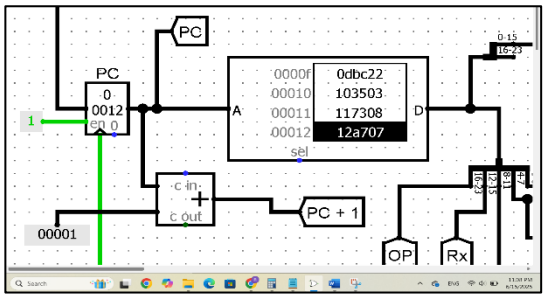


Figure 2.36: Instruction 20 in Memory

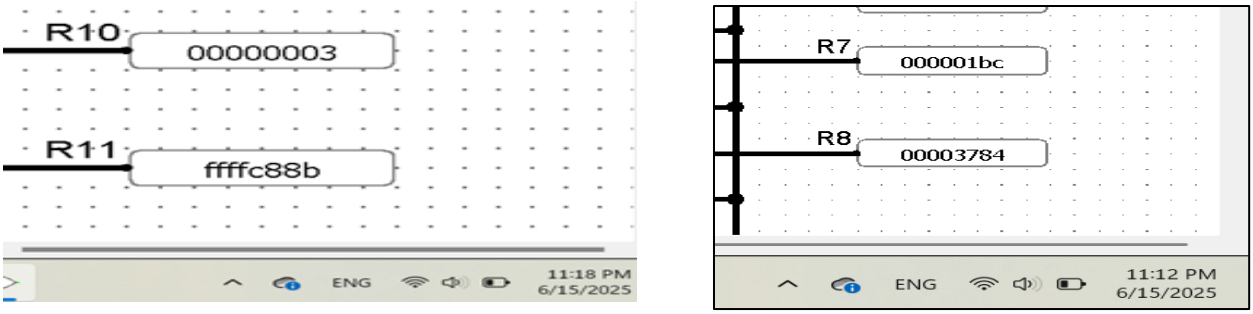


Figure 2.37: R10 and R7 after SRA Operation

2.3.3 Simulating the 21st and the 22nd instructions

Figure 2.38 shows BEQ R7, R12, #0x03 instruction. This instruction branches to the instruction after incrementing the PC by (0x03 <<1) if the values in R7 and R12 are equal, if they are not equal the program continues sequentially.

PC+= (0x03 << 1)

PC+= 0x06

The PC before was 14, R7 = 0x000001bc and R12 = 0x0c00006f as shown in figure 2.39

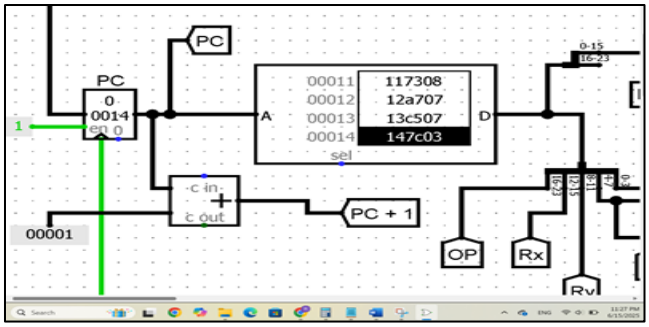


Figure 2.38: Instruction 21 in Memory

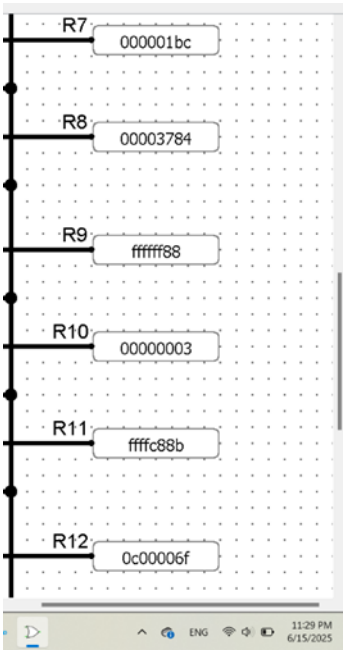


Figure 2.39: Values of Registers in BEQ

Since they are not equal the program will continue sequentially to the next instruction as in figure 2.40.

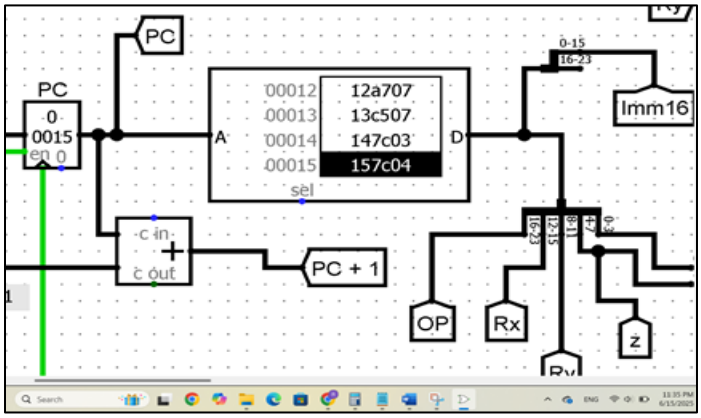


Figure 2.40: Continue Instruction Sequence

Now the next instruction is: BNE R7, R12, #0x04. This instruction branches to the instruction after incrementing the PC by (0x04 <<1) if the values in R7 and R12 are not equal, if they are equal the program continues sequentially.

Since they are not equal, it will branch to (PC+=0x08) which will be 0x01d.

This address contains an instruction 0x021230 which is OR R1, R2, R3, it does bitwise OR between the contents in R2 and R3 and writes the result on R1.

R2 = 0x00000765, R3 = 0x0001bc30

R1 = 0x0001bf75 as shown in figure 2.41

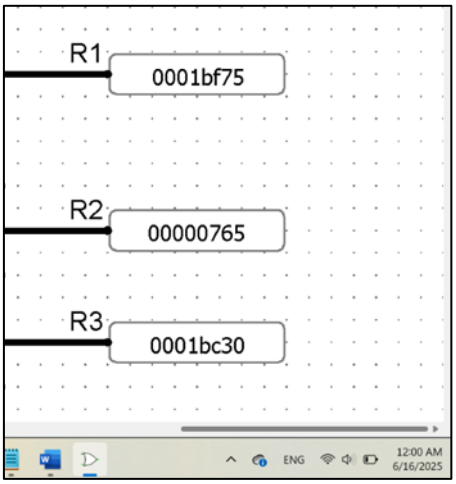


Figure 2.41: R1 and R2 values

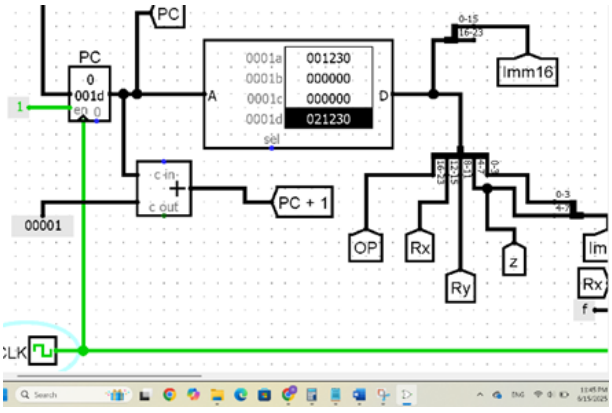


Figure 2.42: Executing the next instruction

2.3.4 Simulating the 26th instruction

Figure 2.43 shows Jump 0x0001 instruction. This jump instruction increments the PC by (0x0001 << 1).

PC+=0x0002

The PC was 0X0001f, it becomes 0x00021 as shown in figures 2.43 and 2.44.

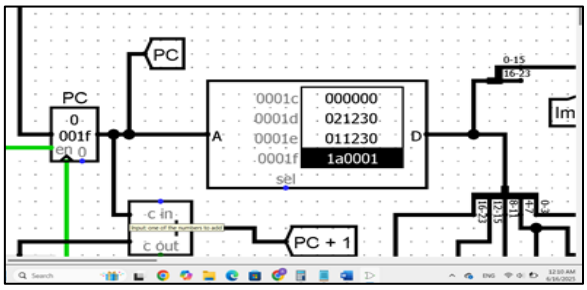


Figure 2.43: Instruction 26

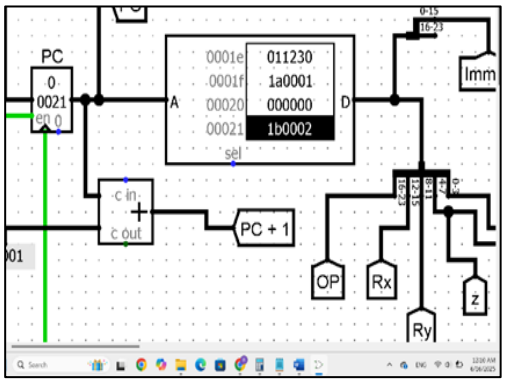


Figure 2.44: Result of Jump Instruction

2.3.5 Simulating the 27th instruction

Figure 2.45 shows JAL 0x0002 instruction. This jump instruction increments the PC by (0x0002 << 1).

PC+=0x0004 and it writes the value PC+1 on R15.

PC = 0x0025

R15 = 0x0022+0x0001=0x0022 as shown in figure 2.46.

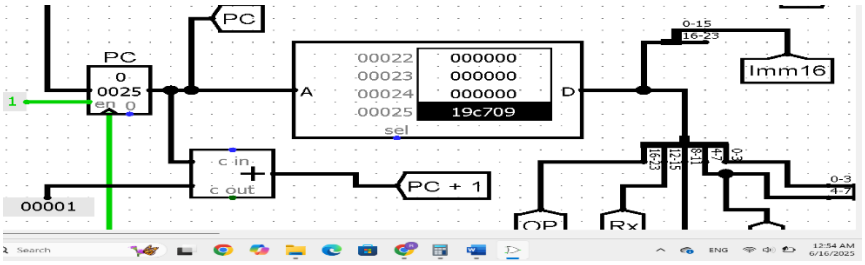


Figure 2.45: Instruction 27

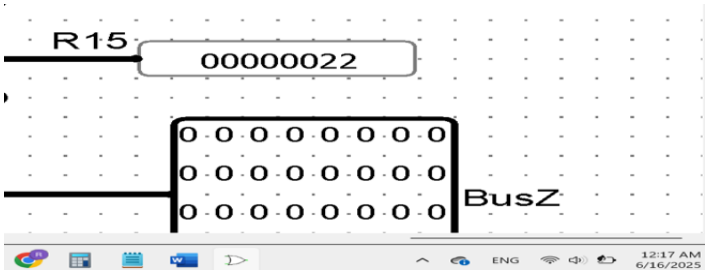


Figure 2.46: R15 after JAL instruction

2.3.6 Simulating the 28th instruction

After the JAL instruction, the instruction that it has jumped to is a store instruction

SW R12, R6, #0x03

MEM[R6+00000011] = R12

R6 = 0x00000000

R12 = 0X42000060 as shown in figure 2.47.

MEM[0x03] = 0x42000060 as shown in figure 2.48

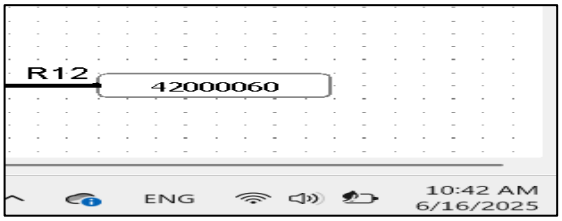


Figure 2.47: R12 value for Store instruction

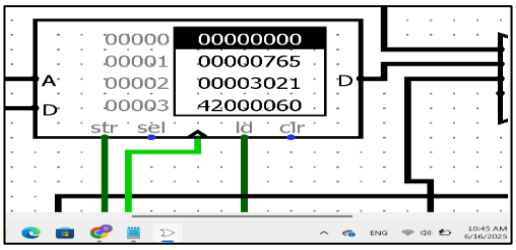


Figure 2.48: Value 0x42000060 stored in RAM

2.4 Testing a simple while loop program

We want to test our program for loops and iterations, so we wrote a simple while loop that increments the integer (i) till its value is equal to the value of integer (n). First, we initialized the register R1 with the value of n which is 0x03, and R2 with value of i which is 0 both using the ANDI instruction as shown below.

R1 = R0 + #0x03 = #0x03

R2 = R0 + #0x00 = #0x00

Then, we started the loop by comparing their values, if they are equal the loop terminates, else it continues sequentially and increments R2 by 1. At the end of the loop the registers should contain the same contents.

int n = 3

int i = 0

while (i != n) {

i = i+1

}

Assembly instruction	Use of it	binary	hexa
ADDI R1, R0, #0x03	Initializing n with the value 3	00001100 0001 0000 00000011	0x0C1003
ADDI R2, R0, #0x00	Initializing i with the value 0	00001100 0010 000 00000000	0x0C2000
BEQ R2, R1, #0x02	Branch if (i==n) PC+=4, else continue sequentially	00010100 0010 0001 00000010	0x142102
ADDI R2, R2, #0X01	Increment the variable i in R2 by 1	00001100 0010 0010 0000 0001	0x0C2201
J #0xFFFF	Jump back to the BEQ instruction that compares the values of i and n. so the Imm8 value is –1 using 2's complement.	0001 1010 1111 1111 1111 1111	0x1AFFFF

Table 2.4: while loop program instructions

The first instruction is ADDI R1, R0, #0x03, therefore, R1 = 0x0003 as shown in figure 2.50.

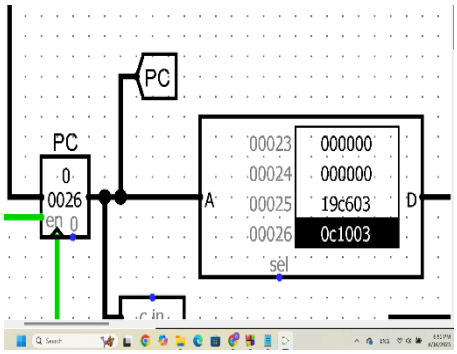


Figure 2.49:instruction 1 from the while loop program

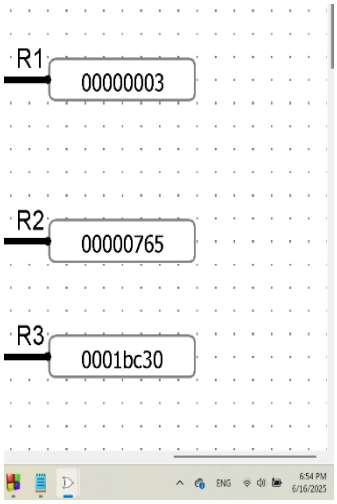


Figure 2.50: R1 set to 3

Second instruction is ADDI R2, R0, #0x00 which is shown in figure 2.51, so R2 is now set to 0 as shown in figure 2.52.

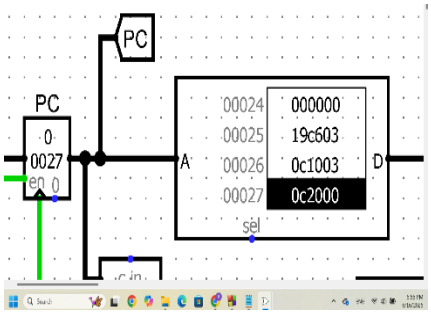


Figure 2.51: instruction 2 in simple while loop program

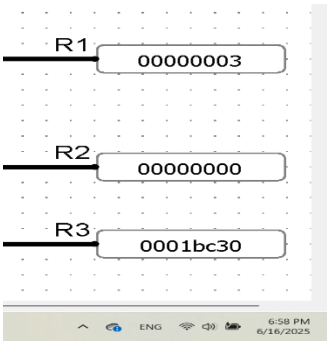


Figure 2.52: R2 set to 0

Third instruction is BEQ R2, R1, #0x01as shown in figure 2.53

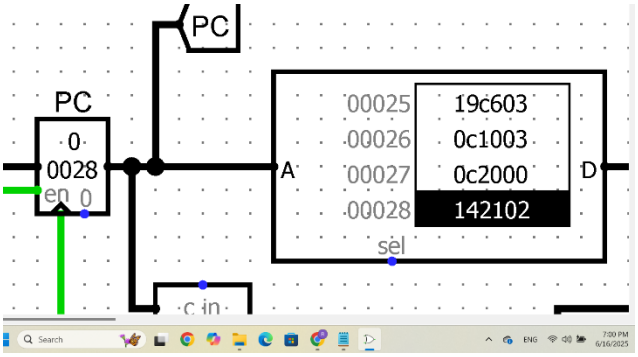


Figure 2.53: instruction 3 in simple while loop program

2.4.1 First iteration:

Since i!=n the pc should increment by one to the next instruction and increment the value of i.

After incrementing i, the jump instruction executes then goes back to the comparing instruction.

Figure 2.54 shows ADDI R2, R2, #0x01 being executed.

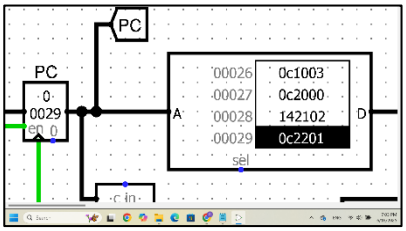


Figure 2.54:instruction 4 in simple while loop program

R2 = R2 + 1 = 0x0001, R2 which is “i” is incremented by one, as shown in figure 2.55.

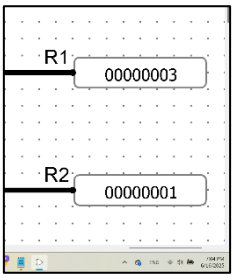


Figure 2.55: R2 incremented by 1

Now in figure 2.56, jump instruction is being executed so that program goes back to check the condition, and figure 2.57 shows that it is back to the BEQ instruction.

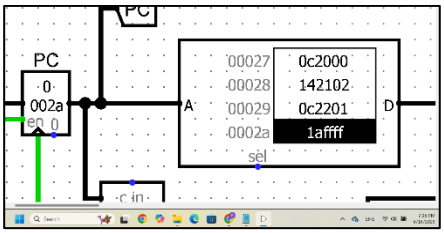


Figure 2.56: instruction 5 in simple while loop program

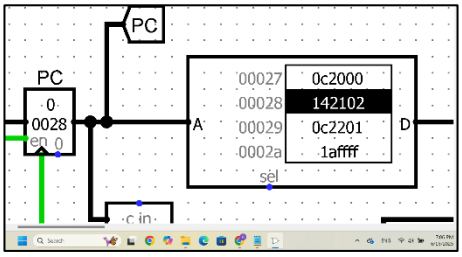


Figure 2.57: going back to BEQ instruction

2.4.2 Second iteration:

Now R2 = 1 and R1 = 3, R2 < R3.

In figure 2.58, the BEQ is executed and since it is false, the program will move to the next following instruction, which is the ADDI instruction.

In figure 2.59, R2 is incremented and becomes 0x0002. Then the jump instruction is executed as shown in figure 2.60.

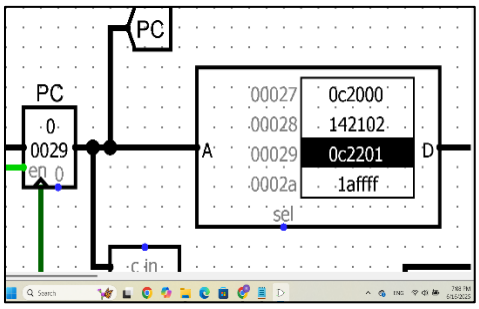


Figure 2.58: going back to instruction 4

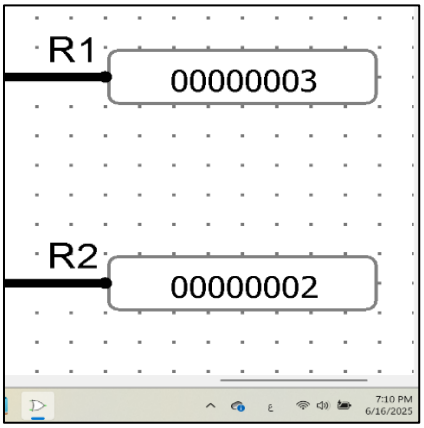


Figure 2.59: R2 is increased for the second time

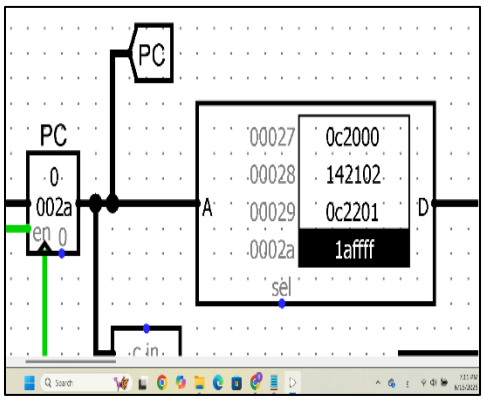


Figure 2.60: reaching jump instruction

2.4.3 Third iteration:

In figure 2.61, the program is currently checking if i!=n and i is currently 2 which does not equal 3, so the program will continue sequentially to the add instruction shown in figure 2.62 and increment i again, so in figure 2.63, R2 = R2 +1 = 3. Then it reaches the jump instruction again as in figure 2.64.

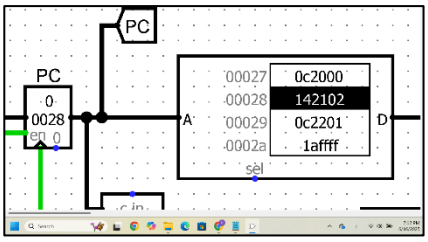


Figure 2.61: checking the BEQ condition again

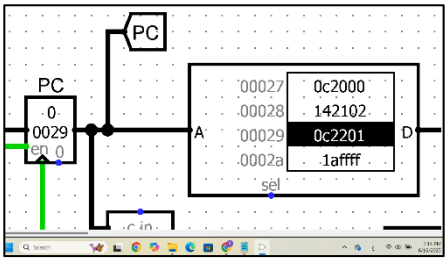


Figure 2.62: executing ADD instruction again

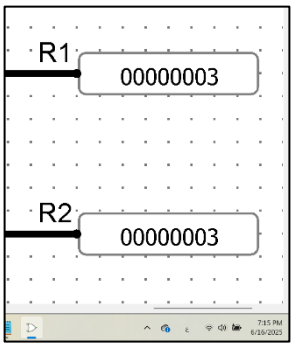


Figure 2.63: R2 increased for the third time

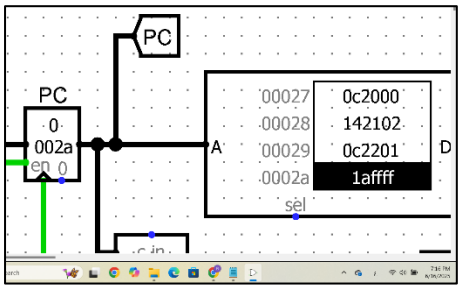


Figure 2.64: reaching jump instruction again

2.4.4 Fourth iteration:

Now in figure 2.65 we are back to checking if i!=n, but currently R2 = 3, and the jump instruction is executed. Then the comparing is done again, but now the condition is true (i = n = 3) so the program exits the loop, and the PC is incremented by 4 as shown in figure 2.66.

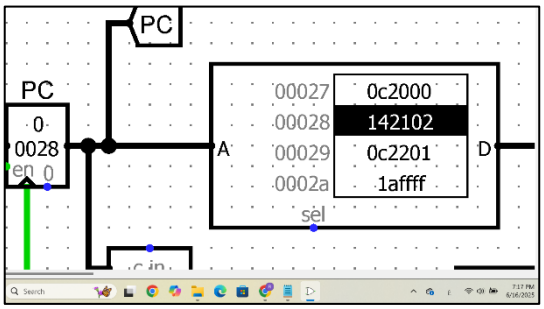


Figure 2.65: checking the BEQ condition for the last time

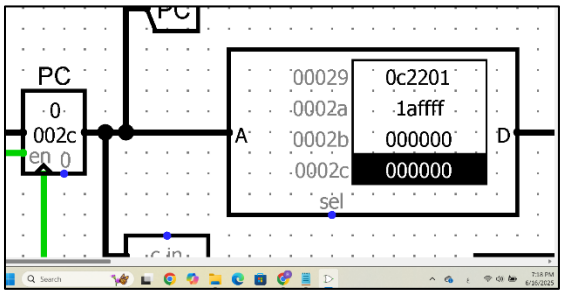


Figure 2.66: exit from the while loop

Based on the simulation results, we can conclude that the program functions correctly, as the while loop behaved exactly as expected: the values of `i` and `n` were updated properly, and the loop terminated when they became equal.

2.5 Testing a procedure and calling it one or more times

In this section, we are aiming to perform an addition operation on three operands R4, R2, and constant 5, then store the result of addition in R14 such that R14=R4+R2+5. However, we clearly know that simple RISC instructions can only add two operands at a clock cycle, not three operands.

Therefore, we will need to calculate R4+R2 using ADD operation and store temporarily result in a general register R13 (R13 = R4 + R2), then use an ADDI operation to add the value in R13 to immediate 5 and store the result in R14 (R14 = R13 + 5), ending up with R14 = R4 + R2 + 5.

Here in our ROM memory, the ADD and ADDI are not sequential in memory address, such that ADDI is stored before ADD instruction, therefore, we will have to use various jump instruction to call ADD instruction first, then call ADDI instruction, and finally jump to the end of the program.

#	Instruction Address	Instruction Format	Instruction Description	Binary	HEX
1	0x0002c	J #x0002	Jump to the add instruction at address 0x00030	00011010 0000 0000 00000010	0x1a0002
2	0x0002d	ADDI R14, R13, #0x05	R14=R13+5	00001100 1110 1101 0000 0101	0xced05
3	0x0002f	J #x0002	Jump to the instruction at address 0x00033 to terminate	00011010 0000 0000 00000010	0x1a0002
4	0x00030	ADD R13, R4, R2	R13=R4+R2	00000100 1101 0100 00100000	0x04d420
5	0x00031	J #xFFFE	Jump to the ADDI instruction at address 0x0002d	00011010 1111 1111 11111110	0x1afffe
6	0x00033	J#0x0000	This instruction terminates the execution of the program by jumping to itself infinitely	00011010 0000 0000 00000000	0x1a0000

Table 2.5: Instruction set for procedure calls

2.5.1 Instruction #1:

In figure 2.67, the program is currently at the J #x0002 instruction at address 0x0002c, however, we know that when a jump instruction is called, the pc will become pc + arithmetic shift left by 1 bit for sign extended Imm8 (same as multiplying by 2), so now pc=pc+shiftLeftBy1Bit(0x0002) =pc+4. This jump instruction aims to jump to the add instruction, so that an add procedure will be executed.

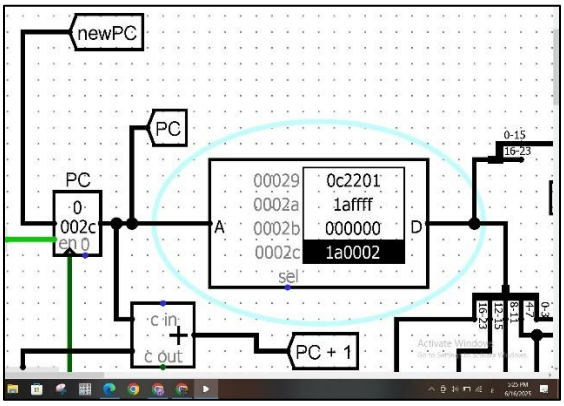


Figure 2.67: Executing first jump instruction in the procedure call instruction set

2.5.2 Calling Instruction #4:

As shown in figure 2.68, the program has jumped to address 0x00030 to call the ADD R13, R4, R2 instruction, so it will now add R2 and R4. The result of the addition will be stored temporarily in R13.

Figure 2.69 shows the values stored in R4 and R2, while figure 2.70 shows R13 which is R4+R2.

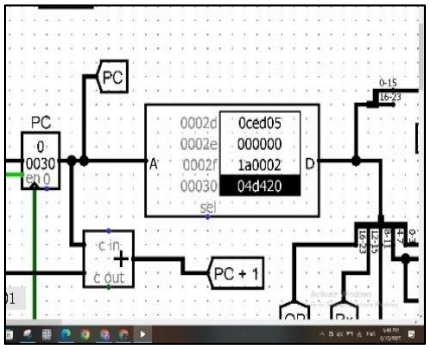


Figure 2.68: Calling instruction #4

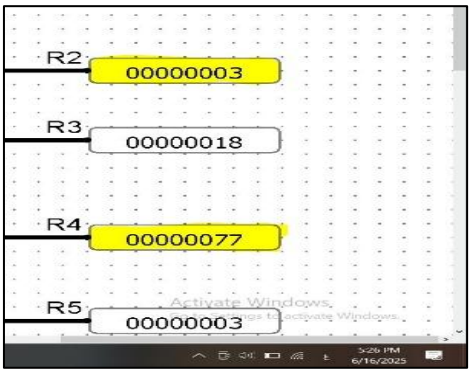


Figure 2.69: R2 and R4 values

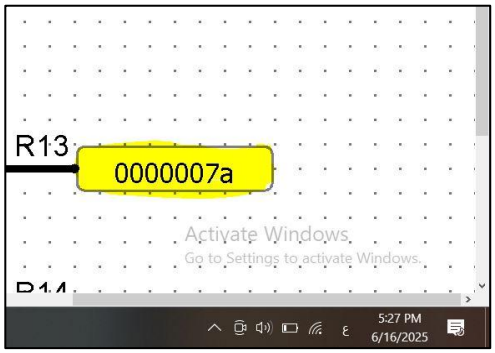


Figure 2.70: R13 after executing instruction #4

2.5.3 Instruction #5

Figure 2.71 shows that our program moved to the following instruction which is at address 0x00031 that contains the instruction J #xFFFE (0x1AFFFE in HEX).

As mentioned above, we know that when a jump instruction is called, the pc will become pc+arithmetic shift left by 1 bit for sign extended Imm8, and here the signed number FFFE represent -2 in decimal, so when shifting it to the left by 1 bit (multiplying it by 2) it will become -4, so our pc=pc-4. Therefore, we will call the ADDI instruction at address 0x0002d.

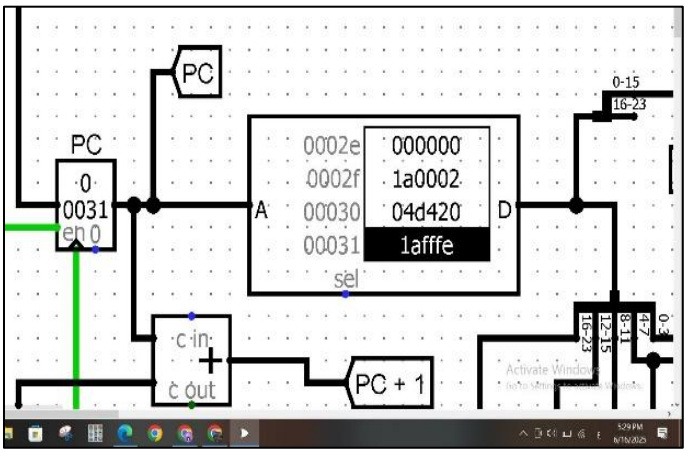


Figure 2.71: Executing instruction #5

2.5.4 Calling Instruction #2

Figure 2.72 shows that our program is now at address 0x0002d because now it should add the temporarily stored value in R13 to immediate 0x05 and return the value in R14, so that we can have the value of R14 that we were aiming to (R14=R13 + 0x05 è R14= R2+R4+0x05).

Figure 2.73 shows the final value of R14.

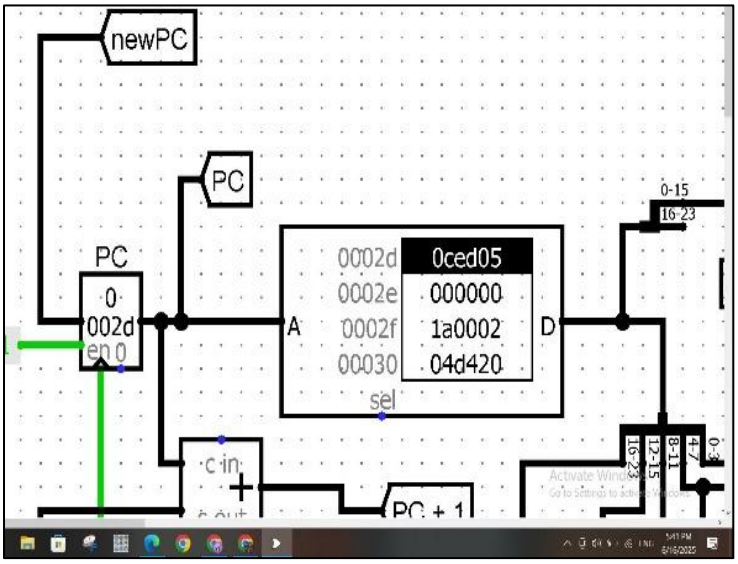


Figure 2.72: Calling instruction #2

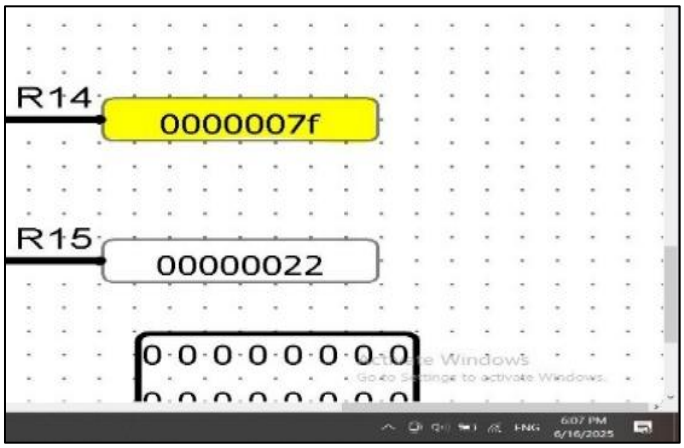


Figure 2.73: R14 after executing instruction #2

2.5.5 Instruction #3

As shown in figure 2.74, our program sequentially moved to the next instruction which is at address 0x0002f and it is another J #x0002 instruction. After executing, pc will become pc+shiftLeftBy1Bit(0x0002)=pc+4 and it will move to address 0x00033 to terminate and end up with having the desired value stored in R14.

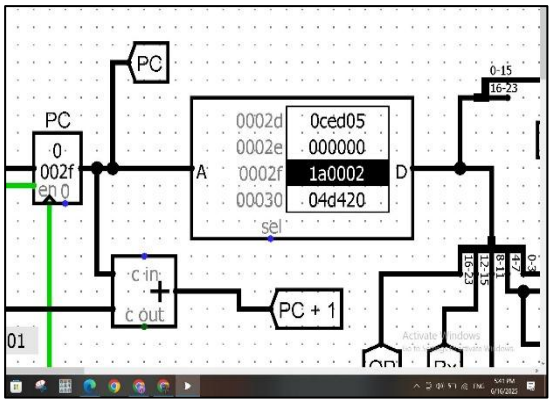


Figure 2.74: Executing instruction #3

2.6 Program Termination

After completing all different instruction sets, especially the last case where we called procedures to calculate the desired value and store it in R14, we now can safely terminate our program by having a jump to itself instruction as the last instruction in our program.

Figure 2.75 shows instruction J #0x0000 (0x1a0000 in HEX). No matter how many times we click on the clock, it will infinitely keep branching to itself because pc=pc+shiftLeftBy1Bit(0x000) =pc+0.

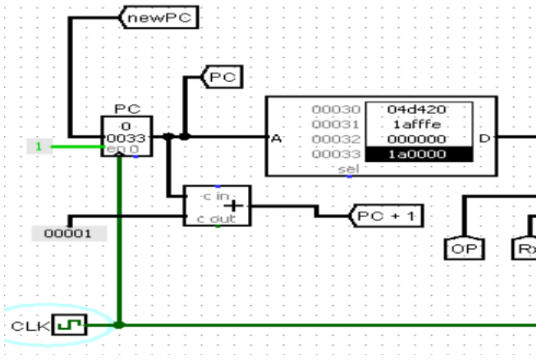


Figure 2.75: Termination Instruction

3. Issues, limitations, and improvements

An encountered issue was that we noticed that the project document has the pc incremented as pc=pc+3 and the instruction memory is byte addressable, however, this can cause two problems. The first is that each instruction is 24 bits (3 bytes) so we will need 3 clock cycles to read an instruction, but this is incompatible with our processor being a single cycle processor. The second problem is that when we thought of using three byte-addressable memories next to each other, each holding 1 byte of the instruction, and reading from all three memories at the same cycle, we realized that it would be a complicated solution in real hardware because it will need additional cost. Therefore, the best solution was to use a memory that is 24-bit wide and increment pc as pc=pc+1.

As for the improvements, we can add a feature to our design to have an option of letting the user choose between starting instructions from address 0x00000 or from a user entered address. It can be implemented by using a 2 to 1 mux as shown in figure 3.1, where selecting 0 means starting from beginning and selecting 1 starts from an input where the desired instruction is stored.

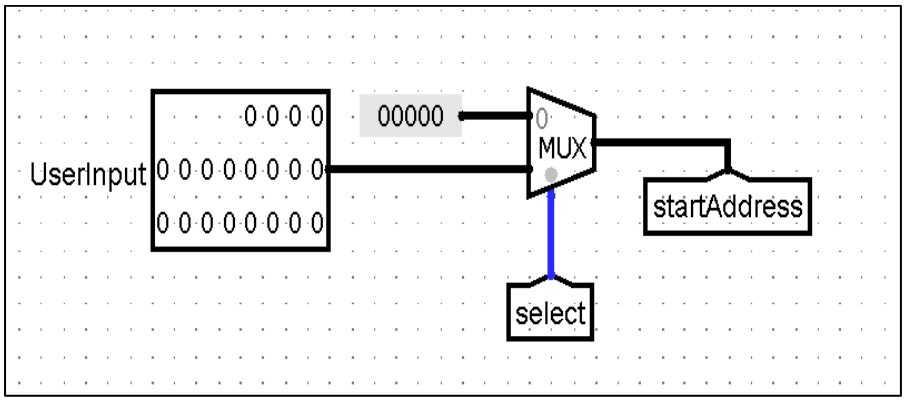


Figure 3.1: Improvement Design

Overall, the experience of using Logisim to design a single cycle processor provided us with a deep understanding of how the processor works, and how all units in a processor are connected to each other. However, it was a challenge to fix common errors and test the final design due to lack of resources on how to trace and execute a program, but we managed to implement all the listed instructions successfully.

4. Teamwork

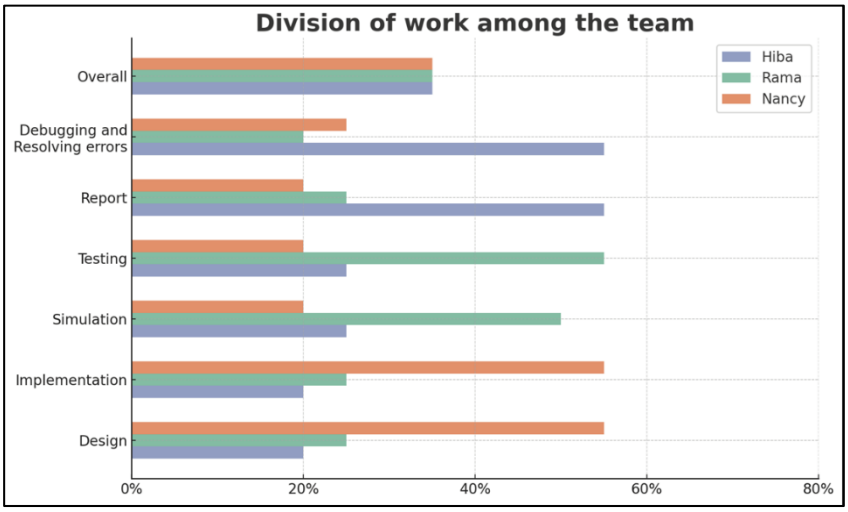


Figure 4.1: Division of work among team

5. Conclusion

We successfully implemented and tested a functional 32-bit simple single cycle processor that we designed using Logism software. The design included implementing the processor datapath which consists of the main control unit, pc control unit, ALU and register files, as well as processor memory and many other simple components such as probes and tunnels.

While implementing each component, we had to deeply understand how it works, for example, we understood how the control unit is responsible for setting up all the control signals that are needed for each instruction to be executed successfully depending on its type: R-type, I-type, and J-type.

After completing the design process, we wrote different assembly codes, then we wrote the machine code for each instruction and stored each instruction in the ROM memory as well as setting some data values in the RAM. Then we fetched each instruction and traced its result, ensuring that each instruction was executed as expected.

At the end of the project, we deepened our understanding of how a processor works at the hardware level, we also had hands-on experience with Logisim software, so it was a great experience that can help us develop more advanced designs in the future.